🧠 AI Evolution Program

Parte 03 — Machine learning clásico

Cubre el aprendizaje a partir de datos antes de las redes profundas, con énfasis en baselines, validación, interpretabilidad y costo de error.

1. 037 — Flujo supervisado y partición train-validation-test
2. 038 — Regresión lineal, regularización y diagnóstico
3. 039 — Clasificación logística y umbrales
4. 040 — Árboles de decisión y reglas interpretables
5. 041 — Random Forest, boosting y ensembles
6. 042 — Ingeniería y selección de características
7. 043 — Clustering y reducción de dimensionalidad
8. 044 — Detección de anomalías
9. 045 — Series temporales y backtesting
10. 046 — Sistemas de recomendación
11. 047 — Métricas, calibración, sesgo y costo de error
12. 048 — Proyecto: producto ML reproducible

037 — Flujo supervisado y partición train-validation-test

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **flujo supervisado y partición train-validation-test** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar flujo supervisado y partición train-validation-test usando los conceptos `supervisado`, `split`, `fuga`, `baseline`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`supervisado`, `split`, `fuga`, `baseline`

🗺️ Ubicación en el mapa de la IA

Con esta clase el programa cruza la frontera hacia el aprendizaje automático: en las partes anteriores el conocimiento se escribía a mano (reglas, distribuciones, heurísticas); aquí el sistema **induce** el mapeo entrada→salida desde ejemplos etiquetados. El protocolo train/validation/test que se establece aquí es el contrato metodológico de *todo* lo que sigue — desde la regresión lineal (clase 038) hasta el fine-tuning de modelos de lenguaje — porque sin partición honesta ninguna métrica del resto del programa es creíble.

📖 Fundamentos

🎓 El problema del aprendizaje supervisado

Dado un conjunto de datos $D = {(x_i, y_i)}_{i=1}^{n}$ con entradas $x_i \in \mathcal{X}$ y etiquetas $y_i \in \mathcal{Y}$, muestreado i.i.d. de una distribución desconocida $P(X, Y)$, el objetivo es encontrar una función $f: \mathcal{X} \to \mathcal{Y}$ que minimice el **riesgo esperado**:

```
R(f) = E[ L(f(X), Y) ]        ← lo que importa: error sobre datos FUTUROS
R̂(f) = (1/n) Σ L(f(xᵢ), yᵢ)   ← lo único medible: error sobre la muestra
```

donde `L` es una función de pérdida (0/1 en clasificación, cuadrática en regresión). El problema central es que optimizamos `R̂` (riesgo empírico) pero queremos `R` (riesgo real). La diferencia `R − R̂` es la **brecha de generalización**, y crece cuando el modelo tiene capacidad suficiente para memorizar la muestra (sobreajuste).

🎀 Por qué tres particiones y no una

- **Train:** ajusta los parámetros del modelo (pesos, umbrales, splits del árbol).
- **Validation (desarrollo):** compara configuraciones e hiperparámetros. Cada vez que se elige "el mejor modelo según validación", la métrica de validación se vuelve optimista, porque la selección misma es una forma de ajuste.
- **Test:** se toca **una sola vez**, al final, para estimar el riesgo real del modelo ya elegido. Si se usa para decidir algo, deja de ser test.

La regla se deriva de un hecho estadístico simple: la estimación de error sobre datos que influyeron en *cualquier* decisión (parámetros, hiperparámetros, features, preprocesado) está sesgada hacia abajo. Proporciones habituales: 60/20/20 u 80/10/10 con datos abundantes; con pocos datos se sustituye validación por **validación cruzada k-fold** (k=5 o k=10): se entrena k veces dejando fuera un pliegue distinto y se promedia.

🕳️ Fuga de datos (data leakage)

Fuga = cualquier información del conjunto de evaluación que se filtra al entrenamiento. Formas típicas, de la más burda a la más sutil:

1. **Duplicados** entre train y test (o el mismo cliente/paciente en ambos lados).
2. **Preprocesado antes del split:** calcular media/desviación para normalizar, imputar nulos o seleccionar features usando TODO el dataset. Lo correcto: ajustar el preprocesador solo con train y aplicarlo a validación/test.
3. **Fuga temporal:** entrenar con datos posteriores a los que se predicen.
4. **Fuga de etiqueta:** una columna que es consecuencia del target (p. ej. "monto reembolsado" para predecir fraude).

📏 Baseline: la vara mínima

Un **baseline** es el modelo más simple razonable: predecir la clase mayoritaria, la media del target, o el valor del instante anterior en series. Cumple dos funciones: (a) detecta problemas triviales o etiquetas filtradas — si el baseline ya acierta 99 %, la tarea o los datos tienen algo raro —, y (b) da denominador al valor del modelo: un accuracy de 0.92 no significa nada si la clase mayoritaria es el 91 % de los casos.

🔁 El flujo completo

```
1. Congelar el protocolo: métrica, split, semilla, baseline.
2. Separar test y NO mirarlo.
3. Ajustar preprocesado + modelo con train.
4. Comparar candidatos con validación (o k-fold).
5. Elegir UN modelo final; reentrenar con train+val si procede.
6. Medir UNA vez sobre test → estimación honesta del riesgo.
7. Reportar: métrica ± incertidumbre, baseline, semilla, versión de datos.
```

🧮 Ejemplo trabajado

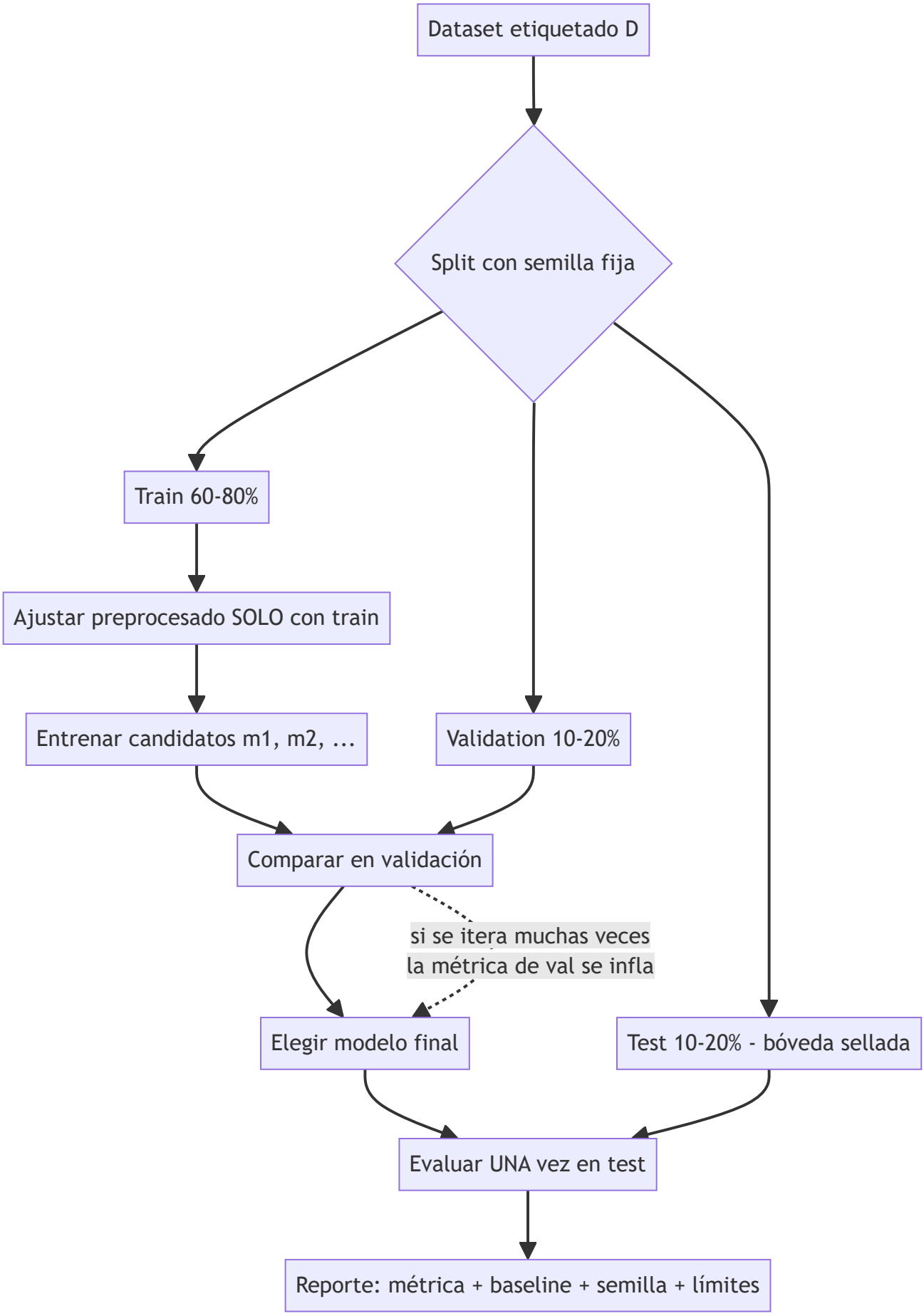
Dataset de 10 correos (5 spam, 5 legítimos) y un clasificador por umbral sobre la cantidad de enlaces del correo. Split 6/2/2 con semilla fija:

Conjunto	Correos (enlaces, etiqueta)
Train (6)	(0,L) (1,L) (2,L) (4,S) (5,S) (7,S)
Val (2)	(1,L) (6,S)
Test (2)	(3,L) (5,S)

Candidatos de umbral `t` (predecir spam si enlaces ≥ t) evaluados en **train**: `t=3` acierta 6/6; `t=2` acierta 5/6 (marca (2,L) como spam); `t=5` acierta 5/6. En **validación**, `t=3` acierta 2/2 y los otros 1/2 o 2/2 según el caso: se elige `t=3`. Recién ahora se mira **test**: (3,L) → 3≥3 predice spam ✗; (5,S) → spam ✓. Accuracy de test = 0.5, igual que el baseline de clase mayoritaria. Moraleja: el 100 % en train y validación era optimismo de muestra pequeña; el test honesto lo revela. Con n tan chico la conclusión correcta es "no hay evidencia de que el umbral supere al baseline", no "el modelo funciona".

📊 Propiedades y comparación

Estrategia de evaluación	Costo de cómputo	Sesgo de la estimación	Varianza	Cuándo usarla
Hold-out simple (train/test)	1 entrenamiento	Bajo si test no se reutiliza	Alta con pocos datos	Datos abundantes
Train/val/test	1 entrenamiento + selección	Val: optimista; test: honesto	Media	Selección de hiperparámetros
k-fold CV (k=5,10)	k entrenamientos	Levemente pesimista	Baja (promedio)	Datos escasos
Leave-one-out	n entrenamientos	Casi insesgado	Alta y caro	n muy pequeño
Split temporal	1+ entrenamientos	Honesto para series	Depende de ventanas	Datos con orden temporal


Dataset etiquetado D
Split con semilla fija
Train 60-80%
Ajustar preprocesado SOLO con train
Entrenar candidatos m1, m2, ...
Validation 10-20%
Comparar en validación
si se itera muchas veces
la métrica de val se infla
Elegir modelo final
Test 10-20% - bóveda sellada
Evaluar UNA vez en test
Reporte: métrica + baseline + semilla + límites

⚠️ Errores conceptuales frecuentes

1. **"Normalizo todo el dataset y después hago el split."** Las estadísticas de normalización ya vieron el test: hay fuga. El orden correcto es split → fit del preprocesador en train → transform del resto.
2. **"Mi accuracy de validación es la estimación del error real."** No: tras comparar muchos candidatos, la métrica del ganador en validación está sesgada al alza. Solo el test intacto estima el riesgo real.
3. **"Puedo mirar el test varias veces si no reentreno."** Cada mirada que influye en una decisión (elegir features, parar el tuning) convierte el test en validación.
4. **"El split aleatorio siempre vale."** Con grupos (mismo paciente, misma tienda) hay que separar por grupo; con series temporales, por tiempo. Un split aleatorio ahí es fuga.
5. **"No necesito baseline, la métrica habla sola."** Sin baseline no se distingue un modelo útil de una clase desbalanceada o una etiqueta filtrada.

🚀 Del aprendizaje a la operación

En producción el dataset no es fijo: la distribución cambia (*drift*), y el protocolo se convierte en monitoreo continuo — reentrenos programados, conjuntos de evaluación refrescados y comparación permanente contra el baseline y el modelo anterior. Faltan además el versionado de datos y de splits (para poder reproducir cualquier métrica histórica), pruebas de fuga automatizadas en el pipeline de features, y la decisión de negocio explícita sobre qué costo tiene cada tipo de error antes de fijar la métrica a optimizar.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- James, Witten, Hastie, Tibshirani — *An Introduction to Statistical Learning* (2e), cap. 2 y 5 (bias-variance y cross-validation), PDF oficial gratuito — uso: desarrollo extendido del tema
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 7 "Model Assessment and Selection", PDF oficial — uso: desarrollo extendido del tema
- scikit-learn User Guide — Cross-validation: evaluating estimator performance — uso: referencia consultada en su fuente original
- scikit-learn — Common pitfalls: data leakage — uso: referencia consultada en su fuente original
- Kaufman et al. (2012), "Leakage in Data Mining: Formulation, Detection, and Avoidance", ACM TKDD. DOI 10.1145/2382577.2382579 — uso: fuente primaria del mecanismo estudiado

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P76 · Un estudio de la validación cruzada y el bootstrap para estimar exactitud y seleccionar modelos	1995	Fija la práctica estándar de evaluación —diez pliegues estratificados— con evidencia empírica en lugar de costumbre.	notebook
P80 · Modelización estadística: las dos culturas	2001	Nombra la división que organiza el campo: suponer un mecanismo generador frente a medir la capacidad de predecir.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 7 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	citada en las referencias de esta clase · cap. 2 y 5 · obra de referencia de la parte 03
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define flujo supervisado y partición train-validation-test sin usar una marca o framework como definición.

2. Explica la relación entre supervisado, split, fuga, baseline.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

038 — Regresión lineal, regularización y diagnóstico

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **regresión lineal, regularización y diagnóstico** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar regresión lineal, regularización y diagnóstico usando los conceptos `regresión`, `L1`, `L2`, `residuos`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`regresión`, `L1`, `L2`, `residuos`

🗺️ Ubicación en el mapa de la IA

La regresión lineal (Legendre 1805, Gauss 1809, formalizada por Galton y Pearson a fines del s. XIX) es el modelo supervisado más antiguo y sigue siendo el punto de partida obligado: es el baseline interpretable contra el que se justifica cualquier modelo más complejo. Introduce tres ideas que atraviesan todo el ML moderno — mínimos cuadrados como optimización de una pérdida, regularización como control de capacidad, y diagnóstico de residuos como auditoría del modelo — y es el ancestro directo de la regresión logística (clase 039) y de cada capa lineal de una red neuronal (parte 04).

📖 Fundamentos

📐 El modelo y la pérdida

El modelo lineal supone que el target es una combinación lineal de las features más ruido:

```
y = β₀ + β₁x₁ + ... + β_d x_d + ε,     ε ~ ruido de media 0
```

Los **mínimos cuadrados ordinarios (OLS)** eligen los coeficientes que minimizan la suma de cuadrados de los residuos:

$$RSS(\beta) = \sum_i (y_i - \hat{y}_i)^2 \quad \text{con} \quad \hat{y}_i = \beta_0 + \sum_j \beta_j x_{ij}$$

En forma matricial, con X de tamaño n×(d+1) (columna de unos incluida), la solución cerrada es la **ecuación normal**:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Para el caso simple (una feature) hay fórmulas directas: $\hat{\beta_1} = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2$ y $\hat{\beta_0} = \bar{y} - \hat{\beta_1}\,\bar{x}$. La calidad del ajuste se resume con $R^2 = 1 - RSS/TSS$, la fracción de varianza del target explicada por el modelo.

🎛️ Regularización: ridge (L2) y lasso (L1)

Cuando hay muchas features, colinealidad o pocos datos, $X^T X$ se vuelve casi singular y los coeficientes OLS explotan en magnitud y varianza. La regularización añade a la pérdida una penalización sobre el tamaño de los coeficientes (nunca sobre β_0):

$$\text{Ridge:} \quad \min RSS(\beta) + \lambda \sum_j \beta_j^2 \qquad \rightarrow \text{encoge todos los coeficientes, no anula ninguno}$$
$$\text{Lasso:} \quad \min RSS(\beta) + \lambda \sum_j |\beta_j| \qquad \rightarrow \text{puede anular coeficientes: selección de variables}$$

- $\lambda = 0$ recupera OLS; $\lambda \to \infty$ colapsa los coeficientes hacia 0 (el modelo tiende a predecir la media).
- λ se elige por validación (o k-fold), nunca sobre el test.
- Ridge tiene solución cerrada $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$; lasso requiere optimización iterativa (descenso por coordenadas) porque $|\beta|$ no es diferenciable en 0.
- La geometría explica la diferencia: la bola L1 tiene esquinas sobre los ejes, y el óptimo restringido cae con probabilidad positiva en una esquina (coeficiente = 0); la bola L2 es esférica y solo encoge.
- **Escalar las features es obligatorio** antes de regularizar: la penalización es la misma para todos los coeficientes, y sus magnitudes dependen de las unidades.

En términos del compromiso sesgo-varianza: OLS es insesgado pero puede tener varianza enorme; la regularización acepta un poco de sesgo a cambio de mucha menos varianza, y el error total (sesgo² + varianza + ruido) suele bajar.

🩺 Diagnóstico de residuos

El residuo $e_i = y_i - \hat{y}_i$ es la ventana a los supuestos del modelo. Se inspecciona el gráfico residuos vs. predicciones y el Q-Q plot:

Patrón en los residuos	Supuesto violado	Acción típica
Curva (forma de U)	Linealidad	Términos polinómicos, transformar features
Abanico (varianza crece)	Homocedasticidad	Transformar y (log), mínimos cuadrados ponderados
Rachas correlacionadas	Independencia	Modelos de series (clase 045)
Colas pesadas en Q-Q	Normalidad del ruido	Pérdidas robustas (Huber), revisar outliers
Puntos aislados con residuo enorme	Outliers / leverage	Investigar el dato antes de borrarlo

Un R^2 alto con residuos estructurados es un modelo equivocado que memoriza la tendencia; un R^2 modesto con residuos aleatorios puede ser el modelo correcto para un fenómeno ruidoso.

🧮 Ejemplo trabajado

Cinco observaciones: horas de estudio x = [1, 2, 3, 4, 5], nota y = [2, 4, 5, 4, 6].

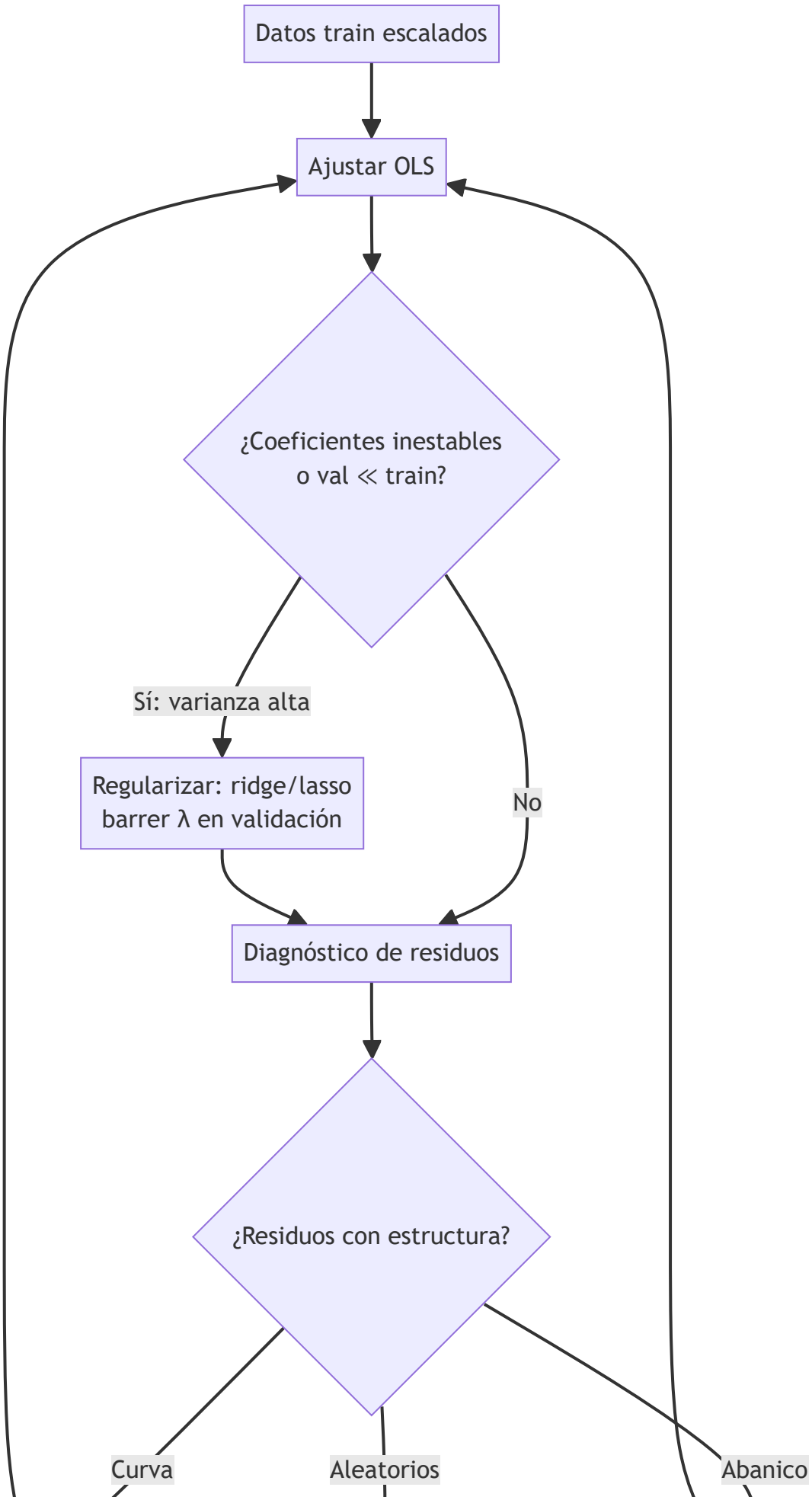
```
x̄ = 3,  ȳ = 4.2
Σ(xᵢ−x̄)(yᵢ−ȳ) = (−2)(−2.2) + (−1)(−0.2) + 0(0.8) + 1(−0.2) + 2(1.8) = 4.4 − 0.2 + 0 − 0.2 + 3.6 = 8.0
Σ(xᵢ−x̄)² = 4 + 1 + 0 + 1 + 4 = 10
β̂₁ = 8.0 / 10 = 0.8        β̂₀ = 4.2 − 0.8·3 = 1.8
```

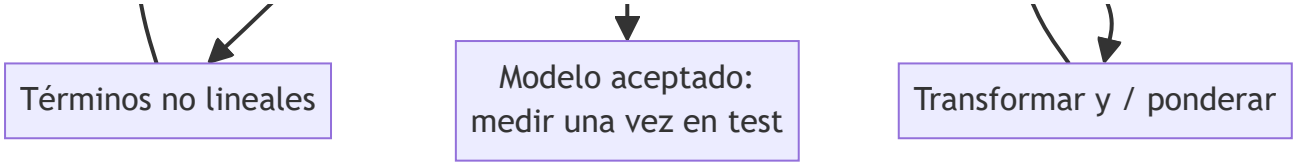
Modelo: $\hat{y} = 1.8 + 0.8x$. Predicciones: [2.6, 3.4, 4.2, 5.0, 5.8]; residuos: [−0.6, 0.6, 0.8, −1.0, 0.2]; RSS = 0.36+0.36+0.64+1.00+0.04 = 2.4; TSS = $\Sigma(y_i-\bar{y})^2$ = 4.84+0.04+0.64+0.04+3.24 = 8.8 → **$R^2 = 1 - 2.4/8.8 \approx 0.727$**.

Versión ridge del coeficiente (con x e y centrados, sin intercepto penalizado): $\hat{\beta}_1(\lambda) = \Sigma x_i y_i \,/\, (\Sigma x_i^2 + \lambda) = 8/(10+\lambda)$. Con $\lambda=2$: $\hat{\beta}_1 = 0.667$ — el coeficiente se encoge hacia 0 y la recta se aplana: menos varianza, algo más de sesgo.

📊 Propiedades y comparación

Método	Solución	Coeficientes nulos	Colinealidad	Hiperparámetro	Cuándo preferirlo
OLS	Cerrada, $O(nd^2 + d^3)$	No	Frágil	Ninguno	$n \gg d$, features poco correlacionadas
Ridge (L2)	Cerrada con λI	No (solo encoge)	Estable	λ por CV	Muchas features correlacionadas
Lasso (L1)	Iterativa	Sí (sparse)	Elige 1 del grupo	λ por CV	Se busca selección de variables
Elastic Net	Iterativa	Sí	Reparte en el grupo	λ, α por CV	Grupos de features correlacionadas


Datos train escalados
Ajustar OLS
¿Coeficientes inestables
o val ≪ train?
Sí: varianza alta
Regularizar: ridge/lasso
barrer λ en validación
No
Diagnóstico de residuos
¿Residuos con estructura?
Curva
Aleatorios
Abanico



⚠️ Errores conceptuales frecuentes

1. **"R² alto = buen modelo."** R² solo mide varianza explicada en los datos usados; sube siempre al añadir features (por eso existe R² ajustado) y no detecta residuos estructurados ni sobreajuste. Se valida fuera de muestra.
2. **"Los coeficientes indican importancia causal."** Un coeficiente mide asociación condicional al resto de las features, con signo que puede invertirse por colinealidad o confusores. Regresión ≠ causalidad (eso exige diseño, clase 035).
3. **"Regularizar sin escalar."** La penalización castiga por igual a un coeficiente en metros y a otro en milímetros; sin estandarizar, λ castiga arbitrariamente según unidades.
4. **"Lasso encontró LAS variables verdaderas."** Con features correlacionadas lasso elige una casi al azar y anula las demás; la selección es inestable entre re-muestreos.
5. **"Más datos siempre arreglan la colinealidad."** La colinealidad exacta (una feature combinación lineal de otras) hace X^TX singular sin importar n; hay que eliminar o combinar features, o regularizar.

🚀 Del aprendizaje a la operación

Entre este núcleo y un uso real median: la elección de λ con validación cruzada anidada (para no contaminar la estimación de error), intervalos de confianza o bootstrap sobre los coeficientes antes de interpretar signos, pruebas de estabilidad del modelo ante re-muestreo, monitoreo de drift de las features en producción (un modelo lineal extrapola linealmente fuera del rango visto, y lo hace en silencio), y la documentación de las transformaciones exactas para reproducir la predicción en el sistema de serving.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- James, Witten, Hastie, Tibshirani — *An Introduction to Statistical Learning* (2e), cap. 3 (regresión lineal) y 6 (regularización), PDF oficial — uso: desarrollo extendido del tema
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 3 "Linear Methods for Regression", PDF oficial — uso: desarrollo extendido del tema

- Tibshirani (1996), "Regression Shrinkage and Selection via the Lasso", JRSS B. DOI 10.1111/j.2517-6161.1996.tb02080.x — uso: fuente primaria del mecanismo estudiado
- Hoerl & Kennard (1970), "Ridge Regression: Biased Estimation for Nonorthogonal Problems", Technometrics. DOI 10.1080/00401706.1970.10488634 — uso: fuente primaria del mecanismo estudiado
- scikit-learn User Guide — Linear Models — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P77 · Contracción y selección en regresión mediante el lasso	1996	Una penalización que estima y selecciona a la vez: pone coeficientes exactamente en cero.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 3 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	citada en las referencias de esta clase · cap. 3 · obra de referencia de la parte 03
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define regresión lineal, regularización y diagnóstico sin usar una marca o framework como definición.
2. Explica la relación entre regresión, L1, L2, residuos.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

039 — Clasificación logística y umbrales

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **clasificación logística y umbrales** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar clasificación logística y umbrales usando los conceptos `logística`, `probabilidades`, `umbral`, `calibración`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`logística`, `probabilidades`, `umbral`, `calibración`

🗺️ Ubicación en el mapa de la IA

La regresión logística (Cox, 1958, sobre la función logística de Verhulst del s. XIX) es el puente entre la regresión lineal de la clase anterior y la clasificación probabilística: mantiene la interpretabilidad del modelo lineal pero produce probabilidades. Su pérdida —la entropía cruzada— y su no-linealidad —la sigmoide— son exactamente las que reaparecen en la neurona de salida de las redes profundas (parte 04). Además introduce la separación conceptual clave de esta parte: **estimar probabilidad** es un problema estadístico; **decidir con un umbral** es un problema de costos.

📖 Fundamentos

🧮 Del score lineal a la probabilidad

La regresión logística modela la probabilidad de la clase positiva pasando un score lineal `z` por la **sigmoide** σ:

```
z = β₀ + β₁x₁ + ... + β_d x_d          (score, en ℝ)
p̂ = σ(z) = 1 / (1 + e^(-z))            (probabilidad, en (0,1))
```

Equivalentemente, el modelo es lineal en el **log-odds** (logit): `log(p/(1-p)) = z`. Cada unidad de aumento en x_j multiplica los odds por `e^{β_j}` — esta es la lectura correcta de los coeficientes (odds ratio), no "sube la probabilidad en β_j".

📉 La pérdida: entropía cruzada (log-loss)

No se ajusta minimizando errores de clasificación (no diferenciable) sino maximizando la verosimilitud de las etiquetas, que equivale a minimizar la **entropía cruzada binaria**:

```
L(β) = -(1/n) Σᵢ [ yᵢ log p̂ᵢ + (1-yᵢ) log(1-p̂ᵢ) ]
```

Propiedades: es convexa (óptimo global único, sin mínimos locales), castiga sin cota las predicciones confiadas y equivocadas ($\hat{p} \to 1$ con $y=0$ cuesta $-\log(1-\hat{p}) \to \infty$), y su gradiente tiene la misma forma que en la regresión lineal:

```
∂L/∂β_j = (1/n) Σᵢ (p̂ᵢ - yᵢ) x_ij      → descenso de gradiente: β_j ← β_j - η ∂L/∂β_j
```

No hay solución cerrada; se optimiza con gradiente o Newton. Con datos linealmente separables los coeficientes divergen a ∞ (la sigmoide quiere ser un escalón): la regularización L2 es también una necesidad numérica.

🎚️ El umbral es una decisión, no parte del modelo

El modelo entrega $\hat{p}$; convertirlo en acción exige un umbral t: predecir positivo si $\hat{p} \geq t$. El t=0.5 por defecto solo es óptimo si los dos errores cuestan lo mismo y las clases están balanceadas. Con matriz de costos explícita, el umbral óptimo que minimiza el costo esperado es:

```
predecir positivo ⇔ p̂ ≥ C_FP / (C_FP + C_FN)
```

donde C_FP es el costo de un falso positivo y C_FN el de un falso negativo. Si un falso negativo cuesta 9 veces más que un falso positivo, $t^* = 1/(1+9) = 0.1$: se acepta disparar muchas alarmas para no dejar pasar casos graves. Mover t recorre el compromiso precision-recall; ninguna elección de t mejora la calidad del score, solo reparte errores.

🌡️ Calibración: que 0.7 signifique 70 %

Un clasificador está **calibrado** si entre los casos con $\hat{p} \approx 0.7$ aproximadamente el 70 % son positivos. La regresión logística bien especificada tiende a estar calibrada (su pérdida es una *proper scoring rule*); árboles, SVM y boosting suelen no estarlo. Se diagnostica con el diagrama de confiabilidad (probabilidad predicha vs. frecuencia observada por bins) y se corrige con **Platt scaling** (una logística sobre los scores) o **regresión isotónica**, ajustadas en validación. La calibración importa porque el umbral por costos de arriba solo es válido si $\hat{p}$ es una probabilidad real.

Ejemplo trabajado

Modelo entrenado: `z = −3 + 1.2·(nº de pagos atrasados)`. Cliente con 3 atrasos:

```
z = −3 + 1.2·3 = 0.6
p̂ = 1/(1 + e^(−0.6)) = 1/(1 + 0.5488) ≈ 0.649
```

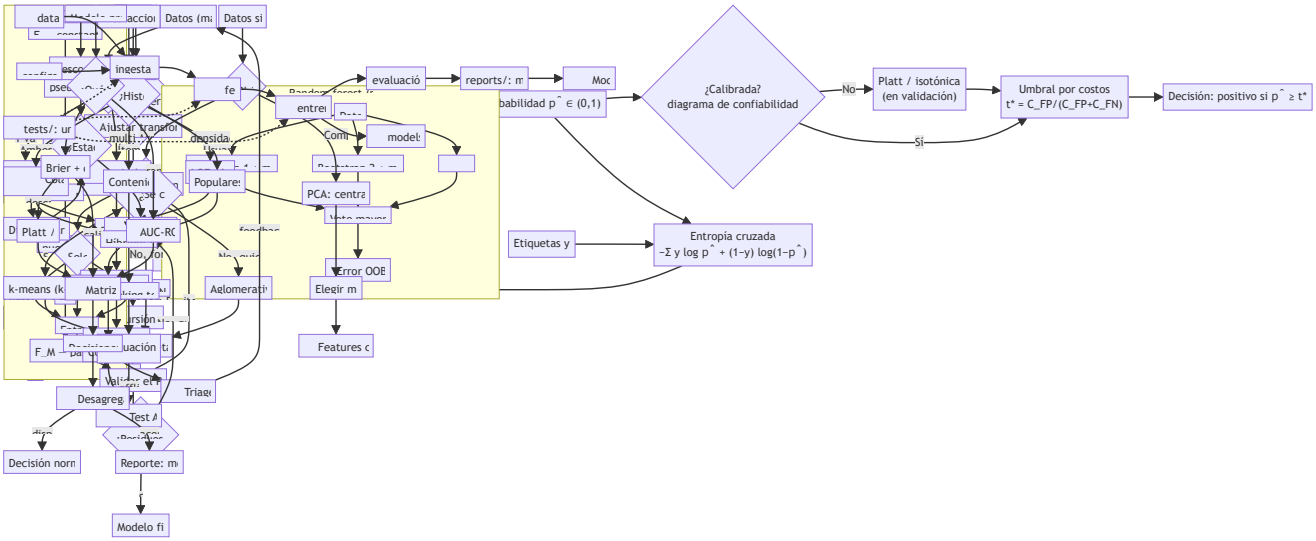
Interpretación del coeficiente: cada atraso multiplica los odds de impago por e^1.2 ≈ 3.32.

Contribución a la pérdida si el cliente finalmente NO impaga (y=0): `−log(1−0.649) = −log(0.351) ≈ 1.047`. Si hubiera impagado (y=1): `−log(0.649) ≈ 0.432`.

Decisión con costos: aprobar a un moroso (FN) cuesta 500; rechazar a un buen cliente (FP) cuesta 100. Umbral óptimo `t* = 100/(100+500) = 1/6 ≈ 0.167`. Como $\hat{p} = 0.649 \geq 0.167$, se rechaza el crédito — con t=0.5 también, pero un cliente con 1 atraso (z=−1.8, $\hat{p} \approx 0.142$) se aprueba con t=0.5 y **también** con t*=0.167 (0.142 < 0.167, por poco): el umbral de costos deja pasar justo a los casos donde el riesgo esperado compensa.

Propiedades y comparación

Aspecto	Regresión logística	Regresión lineal sobre 0/1	Árbol de decisión	k-NN
Salida	Probabilidad calibrable	Valores fuera de [0,1]	Frecuencias por hoja (mal calibradas)	Frecuencia local
Frontera de decisión	Hiperplano	Hiperplano	Cajas alineadas a ejes	Irregular
Pérdida	Entropía cruzada (convexa)	Cuadrática (inadecuada)	Impureza voraz	—
Interpretación	Odds ratio por feature	Pendiente sin sentido probabilístico	Reglas legibles	Ninguna global
Datos separables	Diverge sin L2	—	Sobreajusta hondo	Depende de k



⚠️ Errores conceptuales frecuentes

1. **"El 0.5 es el umbral natural."** Solo si los costos son simétricos y las clases balanceadas; el umbral es una decisión de negocio que se fija con la matriz de costos, después de entrenar.
2. **"β_j es cuánto sube la probabilidad."** β_j actúa sobre el log-odds; el efecto en probabilidad depende del punto (máximo en $\hat{p}=0.5$, casi nulo en los extremos). La lectura correcta es multiplicativa sobre los odds: e^{β_j}.
3. **"Accuracy alta = buen clasificador de probabilidad."** La accuracy solo ve el lado del umbral. Un modelo puede acertar la clase y estar pésimamente calibrado; para scores se evalúa log-loss, Brier o el diagrama de confiabilidad (clase 047).
4. **"La logística es solo para fronteras lineales, así que es débil."** Es lineal en las features *dadas*: con términos polinómicos o interacciones la frontera en el espacio original puede ser curva, manteniendo convexidad e interpretabilidad.
5. **"Si separa perfecto en train, mejor."** Separación perfecta hace diverger los coeficientes y produce $\hat{p} \in \{0,1\}$ sobreconfiadas; señal de que falta regularización o sobran features.

🚀 Del aprendizaje a la operación

Para operar una logística real faltan: recalibración periódica (la calibración se degrada con el drift antes que el ranking), umbrales distintos por segmento cuando los costos difieren, monitoreo de la distribución de scores (un corrimiento del histograma de $\hat{p}$ es la primera alarma de drift), análisis de equidad por subgrupo antes de fijar el umbral (clase 047), y trazabilidad de la versión de modelo + umbral que produjo cada decisión, porque en dominios regulados hay que poder explicar caso por caso.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

Referencias

- James et al. — *An Introduction to Statistical Learning* (2e), cap. 4 "Classification", PDF oficial — uso: desarrollo extendido del tema
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), §4.4 "Logistic Regression", PDF oficial — uso: desarrollo extendido del tema
- Bishop — *Pattern Recognition and Machine Learning* (2006), §4.3 "Probabilistic Discriminative Models", PDF oficial de Microsoft Research — uso: referencia consultada en su fuente original
- Cox (1958), "The Regression Analysis of Binary Sequences", JRSS B. DOI 10.1111/j.2517-6161.1958.tb00292.x — uso: fuente primaria del mecanismo estudiado

- scikit-learn User Guide — Logistic regression — uso: referencia consultada en su fuente original
- scikit-learn User Guide — Probability calibration — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P01 · El perceptrón: un modelo probabilístico de almacenamiento y organización de información en el cerebro	1958	Primera máquina que aprende sus propios pesos a partir de ejemplos en lugar de ejecutar reglas escritas por una persona.	notebook
P75 · Redes de vectores soporte	1995	Convierte la elección entre clasificadores que aciertan igual en un criterio con justificación teórica: el margen.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	citada en las referencias de esta clase · cap. 4 · obra de referencia de la parte 03
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define clasificación logística y umbrales sin usar una marca o framework como definición.
2. Explica la relación entre logística, probabilidades, umbral, calibración.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

040 — Árboles de decisión y reglas interpretables

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **árboles de decisión y reglas interpretables** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar árboles de decisión y reglas interpretables usando los conceptos `árboles`, `impureza`, `poda`, `reglas`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`árboles`, `impureza`, `poda`, `reglas`

🗺️ Ubicación en el mapa de la IA

Los árboles de decisión (CART de Breiman et al. 1984; ID3/C4.5 de Quinlan) unen dos tradiciones del programa: heredan de la IA simbólica (parte 01) la representación por reglas legibles, pero las **inducen desde datos** en lugar de escribirlas a mano. Son el modelo no lineal interpretable de referencia y la pieza básica de los ensembles de la clase siguiente (random forest, boosting), que dominan el ML tabular hasta hoy. Su sesgo inductivo — particiones alineadas a los ejes — explica tanto su legibilidad como sus límites.

📖 Fundamentos

🌳 Qué es un árbol de decisión

Un árbol particiona recursivamente el espacio de features en regiones rectangulares mediante preguntas binarias del tipo $x_j \le t$. Cada hoja predice la clase mayoritaria (o la media, en regresión) de los ejemplos de entrenamiento que caen en ella. Todo camino raíz→hoja es una regla legible: `SI atrasos ≤ 2 Y antigüedad > 3 ENTONCES aprueba`.

🧪 Impureza: qué pregunta elegir

El algoritmo es **voraz**: en cada nodo prueba todos los splits posibles y elige el que más reduce la impureza de los hijos. Para un nodo con proporciones de clase p1,...,p_k:

```
Gini:      G = 1 − Σₖ pₖ²          (0 = puro; máx 0.5 en binario balanceado)
Entropía:  H = −Σₖ pₖ log₂ pₖ      (0 = puro; máx 1 bit en binario balanceado)
```

La ganancia de un split que separa el nodo padre (n ejemplos) en hijos L (n_L) y R (n_R):

```
Δ = I(padre) − (n_L/n)·I(L) − (n_R/n)·I(R)
```

Gini y entropía eligen casi siempre el mismo split (Gini es más barata de computar; la entropía penaliza algo más los nodos mixtos). En regresión, la impureza es la varianza del target en el nodo. El algoritmo se detiene por profundidad máxima, mínimo de ejemplos por hoja, o ganancia mínima.

✂️ Sobreajuste y poda

Un árbol sin restricciones crece hasta hojas puras: memoriza el train (accuracy 1.0) y generaliza mal — es un modelo de **baja sesgo y alta varianza**: cambiar pocos ejemplos puede cambiar el split raíz y con él todo el árbol. Dos remedios:

- **Pre-poda (early stopping):** limitar `max_depth`, `min_samples_leaf`, `min_impurity_decrease` durante la construcción. Barato, pero puede detenerse antes de un split que solo rinde combinado con el siguiente (efecto XOR).
- **Post-poda por costo-complejidad (CART):** crecer el árbol completo y luego minimizar `R_α(T) = R(T) + α·|hojas(T)|`, donde R(T) es el error y α ≥ 0 el precio por hoja. Al aumentar α se colapsan primero las ramas que menos aportan; α se elige por validación cruzada. Es la poda de `ccp_alpha` en scikit-learn.

📜 Reglas interpretables y sus condiciones

La interpretabilidad del árbol es real pero condicionada: (a) vale para árboles pequeños (≤ ~7 niveles; con 50 hojas nadie "lee" el modelo); (b) la importancia de features por reducción de impureza está sesgada hacia variables con muchos valores posibles y se reparte arbitrariamente entre features correlacionadas; (c) las reglas son fieles al modelo, no al fenómeno: describen cómo decide el árbol, no por qué ocurre el resultado.

🧮 Ejemplo trabajado

Nodo raíz con 10 solicitudes de crédito: 6 aprobadas (A) y 4 rechazadas (R).

```
Gini(raíz) = 1 − (0.6² + 0.4²) = 1 − 0.52 = 0.48
```

Split candidato 1: `atrasos ≤ 1` → izquierda: 5 ejemplos (5A, 0R); derecha: 5 (1A, 4R).

```
Gini(izq) = 1 − (1² + 0²) = 0
Gini(der) = 1 − (0.2² + 0.8²) = 1 − 0.68 = 0.32
Δ₁ = 0.48 − (5/10)·0 − (5/10)·0.32 = 0.48 − 0.16 = 0.32
```

Split candidato 2: `ingreso ≤ 30k` → izquierda: 4 (2A, 2R); derecha: 6 (4A, 2R).

```
Gini(izq) = 1 − (0.5² + 0.5²) = 0.5
Gini(der) = 1 − ((4/6)² + (2/6)²) = 1 − (0.444 + 0.111) = 0.444
Δ₂ = 0.48 − 0.4·0.5 − 0.6·0.444 = 0.48 − 0.2 − 0.267 = 0.013
```

Gana `atrasos ≤ 1` ($\Delta = 0.32 \gg 0.013$). La rama izquierda ya es pura (hoja "aprueba"); la derecha (1A, 4R) puede volver a dividirse o quedar como hoja "rechaza" con confianza 4/5. Regla resultante: `SI atrasos ≤ 1 → aprueba; SI atrasos > 1 → rechaza (80 %)`.

📊 Propiedades y comparación

Aspecto	Árbol CART	Regresión logística	k-NN	Reglas a mano (parte 01)
Frontera	Cajas alineadas a ejes	Hiperplano	Irregular local	La que el experto escriba
No linealidad / interacciones	Automáticas	Manuales	Implícitas	Manuales
Preprocesado	No exige escalar	Exige escalar (con L2)	Exige escalar	—
Varianza	Alta (inestable)	Baja	Media	Nula (fijas)
Interpretación	Reglas si es pequeño	Odds ratio	Ninguna global	Total
Fronteras oblicuas	Escalera de splits	Nativas	Aproximadas	—

⚠️ Errores conceptuales frecuentes

1. **"El árbol encontró el split óptimo global."** El algoritmo es voraz nodo a nodo; el árbol globalmente óptimo es NP-completo. Un split localmente mediocre puede habilitar splits excelentes después (XOR), y el voraz se lo pierde.
2. **"Accuracy 1.0 en train: el árbol es buenísimo."** Es la firma del sobreajuste: sin restricciones el árbol memoriza. La cifra relevante es validación, y la brecha train-val mide la varianza del modelo.
3. **"La importancia de features del árbol identifica las causas."** La importancia por impureza favorece features de alta cardinalidad y se la reparten al azar las features correlacionadas; además describe al modelo, no al fenómeno.

4. **"Los árboles no necesitan nada de preprocesado."** No exigen escalar, pero sí sufren con etiquetas ruidosas, clases desbalanceadas (el split mayoritario domina) y extrapolación: fuera del rango de train predicen la hoja del borde, constante.
5. **"Poda = perder accuracy."** Pierde accuracy de *train* y suele ganar en validación: la poda cambia varianza por sesgo, exactamente el intercambio correcto en un modelo inestable.

🚀 Del aprendizaje a la operación

Para llevar un árbol a decisiones reales faltan: elegir α (o profundidad) con validación anidada y verificar la estabilidad del árbol ante re-muestreo (si cada bootstrap da reglas distintas, no comuniques "las reglas" como conocimiento), auditar las reglas con expertos del dominio antes de automatizar (una regla legible también puede codificar un sesgo legible), definir el fallback para valores nulos o fuera de rango en producción, y — si la prioridad es exactitud sobre legibilidad — pasar al ensemble de la clase 041, aceptando el costo en interpretabilidad.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Breiman, Friedman, Olshen, Stone — *Classification and Regression Trees* (1984), el libro de CART. DOI 10.1201/9781315139470 — uso: fuente primaria del mecanismo estudiado
- Quinlan (1986), "Induction of Decision Trees", *Machine Learning* 1. DOI 10.1007/BF00116251 — uso: fuente primaria del mecanismo estudiado
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), §9.2 "Tree-Based Methods", PDF oficial — uso: desarrollo extendido del tema
- James et al. — *An Introduction to Statistical Learning* (2e), cap. 8 "Tree-Based Methods", PDF oficial — uso: desarrollo extendido del tema
- scikit-learn User Guide — Decision Trees (incluye poda por costo-complejidad) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P74 · Inducción de árboles de decisión	1986	Aprende un modelo que una persona puede leer, eligiendo cada pregunta por cuánta incertidumbre elimina.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	citada en las referencias de esta clase · cap. 8 · obra de referencia de la parte 03
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define árboles de decisión y reglas interpretables sin usar una marca o framework como definición.
2. Explica la relación entre árboles, impureza, poda, reglas.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

041 — Random Forest, boosting y ensembles

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **random forest, boosting y ensembles** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar random forest, boosting y ensembles usando los conceptos `ensembles`, `bagging`, `boosting`, `diversidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`ensembles`, `bagging`, `boosting`, `diversidad`

🗺️ Ubicación en el mapa de la IA

Los ensembles resuelven la debilidad central del árbol de la clase anterior — su varianza — combinando muchos modelos imperfectos: bagging y random forest (Breiman, 1996 y 2001) los promedian en paralelo; boosting (Freund & Schapire 1997; Friedman 2001) los encadena en secuencia. Sus descendientes de gradiente (XGBoost, LightGBM) siguen siendo el estado del arte en datos tabulares, compitiendo de igual a igual con las redes profundas de la parte 04. La idea de que "muchos débiles coordinados superan a uno fuerte" reaparece luego en mixture-of-experts y en las votaciones de self-consistency de los LLM.

📖 Fundamentos

🎲 Por qué promediar reduce varianza

Si B estimadores tienen cada uno varianza σ² y correlación media ρ entre sí, la varianza del promedio es:

```
Var(promedio) = ρσ² + (1−ρ)σ²/B
```

Con B → ∞ el segundo término desaparece, pero el primero **no**: el techo de mejora lo pone la correlación entre los modelos. Todo el diseño de un ensemble por promedio consiste en fabricar modelos individualmente decentes y mutuamente **descorrelacionados**.

👜 Bagging y random forest

- **Bagging (bootstrap aggregating):** entrenar B árboles profundos, cada uno sobre una muestra bootstrap (n ejemplos con reemplazo; cada muestra deja fuera ≈ 36.8 % de los datos, pues $(1-1/n)^n \to e^{-1} \approx 0.368$). Predicción: voto mayoritario (clasificación) o media (regresión).
- **Random forest = bagging + descorrelación extra:** en cada nodo, el árbol solo puede elegir el split entre un subconjunto aleatorio de `m` features (típico: $m = \sqrt{d}$ en clasificación, $d/3$ en regresión). Esto evita que todos los árboles empiecen por la misma feature dominante, bajando ρ.
- **Error OOB (out-of-bag):** cada ejemplo se evalúa con los árboles que no lo vieron en su bootstrap (~37 % de ellos): una validación "gratis" casi equivalente a cross-validation.
- Aumentar B nunca sobreajusta por sí mismo (solo estabiliza el promedio); el costo es cómputo y memoria.

🚀 Boosting: sumar correctores en secuencia

Boosting construye un modelo aditivo por etapas, donde cada modelo nuevo corrige los errores del acumulado:

```
F_M(x) = F₀ + Σ_{m=1..M} ν · h_m(x)     h_m: árbol pequeño (stump o profundidad 2-4)
```

- **AdaBoost (1997):** re-pondera ejemplos; los mal clasificados pesan más en la ronda siguiente. El peso de cada árbol es $\alpha_m = \tfrac{1}{2}\cdot\ln((1-\mathrm{err}_m)/\mathrm{err}_m)$ y los pesos de los ejemplos fallados se multiplican por e^{α_m}. Equivale a minimizar la pérdida exponencial por etapas.
- **Gradient boosting (2001):** generaliza a cualquier pérdida diferenciable: en cada etapa se ajusta un árbol a los **pseudo-residuos** (gradiente negativo de la pérdida respecto de la predicción actual). Con pérdida cuadrática el pseudo-residuo es literalmente el residuo `y - F(x)`.
- El *learning rate* ν (0.01-0.3) encoge cada aporte; ν pequeño + más árboles suele generalizar mejor. A diferencia del forest, **boosting sí sobreajusta** con M grande: M se elige con early stopping en validación.

En términos del compromiso sesgo-varianza: bagging ataca la **varianza** (promedia modelos de bajo sesgo), boosting ataca el **sesgo** (suma modelos débiles que se especializan en lo que falta) controlando la varianza con ν, la profundidad del débil y el submuestreo.

🧮 Ejemplo trabajado

Voto de mayoría: 5 clasificadores independientes, cada uno con accuracy 0.7. El ensemble por mayoría acierta si aciertan al menos 3:

```
P(3 de 5) = C(5,3)·0.7³·0.3² = 10·0.343·0.09  = 0.3087
P(4 de 5) = C(5,4)·0.7⁴·0.3¹ =  5·0.2401·0.3  = 0.3602
P(5 de 5) = 0.7⁵                              = 0.1681
P(mayoría acierta) = 0.3087 + 0.3602 + 0.1681 ≈ 0.837
```

Cinco modelos del 70 % → ensemble del 83.7 %, **si los errores son independientes**. Si los cinco fueran clones (ρ = 1) el ensemble seguiría en 0.7: la diversidad lo es todo.

Una ronda de AdaBoost: 10 ejemplos con peso 1/10; el stump h_1 falla en 2 → $\text{err}_1 = 0.2$ y $\alpha_1 = \frac{1}{2}\cdot\ln(0.8/0.2) = \frac{1}{2}\cdot\ln 4 \approx 0.693$. Los 2 fallados multiplican su peso por $e^{0.693} \approx 2$ (pasan a 0.2) y los 8 acertados por $e^{-0.693} \approx 0.5$ (pasan a 0.05); la suma es $2\cdot0.2 + 8\cdot0.05 = 0.8$ y tras renormalizar cada fallado pesa 0.25 y cada acertado 0.0625. La ronda 2 queda obligada a ocuparse de los casos difíciles.

📊 Propiedades y comparación

Aspecto	Árbol único	Random forest	AdaBoost	Gradient boosting
Ataca principalmente	—	Varianza	Sesgo	Sesgo (pérdida flexible)
Entrenamiento	1 árbol	Paralelo (B árboles)	Secuencial	Secuencial
Sobreajuste al crecer B/M	—	No (satura)	Sí (moderado)	Sí (exige early stopping)
Ruido de etiqueta	Media	Baja	Alta (re-pondera errores)	Media (según pérdida)
Hiperparámetros clave	profundidad, α	B, m features/nodo	M, profundidad del débil	M, ν, profundidad, submuestreo
Interpretabilidad	Alta (pequeño)	Baja (importancias)	Baja	Baja
Validación interna	—	OOB gratis	No	Early stopping en val

⚠️ Errores conceptuales frecuentes

1. **"Más árboles en el forest terminarán sobreajustando."** No: B solo estabiliza el promedio; el error converge a un límite fijado por ρ y la calidad de cada árbol. Lo que sí sobreajusta es M en boosting.
2. **"El ensemble siempre supera a sus miembros."** Solo si los miembros son mejores que el azar y sus errores están (parcialmente) descorrelacionados. Promediar clones no aporta nada; promediar modelos malos promedia basura.
3. **"Random forest y boosting son intercambiables."** Atacan errores opuestos: forest promedia modelos de bajo sesgo para bajar varianza; boosting encadena modelos de alto sesgo para bajarlo. Con

etiquetas ruidosas el forest suele ser más robusto; con señal compleja y datos limpios, el boosting suele ganar.

4. **"La importancia de features del forest es fiable."** Hereda los sesgos del árbol (cardinalidad, correlación); la importancia por permutación en OOB/validación es preferible, y ninguna implica causalidad.
5. **"Las probabilidades del boosting son probabilidades."** Los scores de boosting suelen estar descalibrados (demasiado extremos AdaBoost, depende de la pérdida en GB); antes de aplicar umbrales por costo hay que calibrar (clase 039 y 047).

🚀 Del aprendizaje a la operación

Para operar un ensemble real faltan: búsqueda de hiperparámetros con presupuesto explícito (ν, M, profundidad interactúan; early stopping en validación separada), calibración de probabilidades antes de decidir con umbrales, importancia por permutación y ejemplos contrafactuales para explicar decisiones (obligatorio en dominios regulados), control del costo de inferencia (500 árboles × profundidad 12 tienen latencia y memoria reales), y monitoreo de drift: el ensemble extrapola constante fuera del rango visto, igual que sus árboles.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Breiman (2001), "Random Forests", *Machine Learning* 45. DOI 10.1023/A:1010933404324 — uso: fuente primaria del mecanismo estudiado
- Breiman (1996), "Bagging Predictors", *Machine Learning* 24. DOI 10.1007/BF00058655 — uso: fuente primaria del mecanismo estudiado
- Freund & Schapire (1997), "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting", JCSS. DOI 10.1006/jcss.1997.1504 — uso: fuente primaria del mecanismo estudiado
- Friedman (2001), "Greedy Function Approximation: A Gradient Boosting Machine", *Annals of Statistics* 29(5). DOI 10.1214/aos/1013203451 — uso: fuente primaria del mecanismo estudiado

- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 10 (boosting) y 15 (random forests), PDF oficial — uso: desarrollo extendido del tema
- scikit-learn User Guide — Ensembles: bagging, forests, AdaBoost, gradient boosting — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P78 · Una generalización decisional del aprendizaje en línea y su aplicación al boosting	1997	Demuestra que muchos clasificadores apenas mejores que el azar se combinan en uno arbitrariamente bueno, y da el algoritmo que lo hace.	notebook
P79 · Bosques aleatorios	2001	Demuestra que el error de un conjunto depende de la fuerza de sus miembros Y de su correlación, y que empeorarlos a propósito puede mejorarlo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 10 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define random forest, boosting y ensembles sin usar una marca o framework como definición.
2. Explica la relación entre ensembles, bagging, boosting, diversidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

042 — Ingeniería y selección de características

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ingeniería y selección de características** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ingeniería y selección de características usando los conceptos `features`, `encoding`, `selección`, `pipelines`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`features`, `encoding`, `selección`, `pipelines`

🗺️ Ubicación en el mapa de la IA

En el ML clásico, la representación de los datos importa más que la elección del modelo: "applied machine learning is basically feature engineering" (Andrew Ng). Esta clase sistematiza cómo convertir datos crudos en features útiles y cómo elegir cuáles conservar — con el protocolo anti-fuga de la clase 037 como restricción permanente. Es también el contraste que da sentido al deep learning (parte 04), cuya promesa central es justamente **aprender** las representaciones que aquí se construyen a mano.

📖 Fundamentos

🔢 Transformaciones numéricas

- **Estandarización:** `z = (x − μ)/σ` (media 0, desviación 1). Necesaria para modelos basados en distancias o con regularización (k-NN, SVM, ridge/lasso, redes).
- **Min-max:** `x' = (x − min)/(max − min)` a [0,1]; sensible a outliers.
- **Log / Box-Cox:** comprime colas largas (ingresos, conteos); convierte relaciones multiplicativas en aditivas.

- **Discretización (binning):** convierte una numérica en categorías (edad → tramos); pierde información pero captura no linealidades en modelos lineales.
- Los árboles y ensembles son invariantes a transformaciones monótonas de cada feature: escalar no les afecta; el log tampoco (mismo orden ⇒ mismos splits).

🏷️ Codificación de categóricas

- **One-hot:** una columna binaria por categoría. Seguro pero explota en cardinalidad alta (10 000 códigos postales → 10 000 columnas).
- **Ordinal:** entero por categoría; solo válido si existe orden real (S < M < L). Un orden inventado introduce estructura falsa en modelos lineales/distancia.
- **Target encoding:** sustituir la categoría por la media del target en esa categoría. Potente en alta cardinalidad y **la fuente de fuga más clásica**: debe calcularse con esquema out-of-fold (la media que ve cada fila se calcula sin esa fila) y con suavizado hacia la media global para categorías raras: `enc(c) = (n_c·ȳ_c + k·ȳ_global)/(n_c + k)`.
- **Hashing:** proyecta categorías a un número fijo de columnas; admite categorías nuevas a costa de colisiones.

🧩 Interacciones, fechas y nulos

- **Interacciones:** productos o razones de features (`deuda/ingreso`); los modelos lineales no las descubren solos.
- **Fechas:** descomponer en día de semana, mes, festivo, tiempo desde el último evento. Para variables cíclicas (hora, mes) usar `sin(2πt/T), cos(2πt/T)` para que las 23 h y las 0 h queden cerca.
- **Nulos:** imputar (media/mediana ajustada en train, o por modelo) + una columna indicadora `was_missing`, porque la ausencia misma suele ser señal. Nunca imputar con estadísticas del dataset completo (fuga).

🎯 Selección de características

Tres familias, de más barata a más fiel al modelo final:

1. **Filtro:** puntuar cada feature contra el target sin modelo (correlación, información mutua, test χ^2). Rápido, ignora interacciones y redundancia.
2. **Wrapper:** buscar subconjuntos entrenando el modelo (selección hacia adelante/atrás, RFE). Fiel pero caro y propenso a sobreajustar la búsqueda: exige CV anidada.
3. **Embedded:** la selección ocurre dentro del entrenamiento — lasso (coeficientes a 0), importancias de árboles con umbral. Buen compromiso costo/calidad.

Regla anti-fuga transversal: **toda** transformación con parámetros aprendidos (μ, σ, medias de target, vocabularios, features seleccionadas) se ajusta SOLO con train, dentro del pipeline que se valida — seleccionar features con el dataset completo y "luego hacer CV" infla la métrica de manera sistemática (el *selection bias* clásico descrito en ESL §7.10.2).

🧮 Ejemplo trabajado

Target encoding con suavizado (k = 10) para la feature `ciudad` prediciendo impago (media global $\bar{y} = 0.10$):

Ciudad	n_c	impagos	ȳ_c	enc = (n·ȳ_c + 10·0.10)/(n+10)
A	90	18	0.20	(90·0.20 + 1)/(103) = 0.19
B	40	2	0.05	(40·0.05 + 1)/50 = 0.06
C	2	2	1.00	(2·1.00 + 1)/12 = 0.25

Sin suavizado, la ciudad C (2 casos, ambos impagos) quedaría codificada como 1.00 — memorización pura de dos filas. El suavizado la arrastra hacia la media global: 0.25. Y aun así, si este encoding se calcula sobre TODO el dataset, cada fila de C "conoce" su propia etiqueta a través del encoding: en CV se vería un lift ficticio. Con esquema out-of-fold, las filas de C se codifican usando solo las *otras* filas de C del train.

📊 Propiedades y comparación

Técnica	Cardinalidad alta	¿Aprende del target?	Riesgo de fuga	Modelos que la exigen
One-hot	Mala (explota)	No	Bajo	Lineales, distancia
Ordinal	Buena	No	Bajo (pero orden falso)	Solo con orden real
Target encoding	Excelente	Sí	**Alto** (out-of-fold obligatorio)	Cualquiera
Hashing	Excelente	No	Bajo (colisiones)	Lineales online
Filtro (corr/MI)	—	Sí (por feature)	Medio (hacerlo en train)	—
Wrapper (RFE)	—	Sí (por subconjunto)	Alto sin CV anidada	—
Embedded (lasso)	—	Sí	Bajo dentro del pipeline	—

⚠️ Errores conceptuales frecuentes

1. **"Selecciono features con todo el dataset y después valido el modelo."** El sesgo de selección contamina la CV: las features ya vieron el target de las filas de validación. La selección va DENTRO del pipeline validado.
2. **"Target encoding = reemplazar por la media y listo."** Sin out-of-fold ni suavizado es una copia parcial del target dentro de las features: lift enorme en validación, colapso en producción.
3. **"Escalar siempre ayuda."** A los árboles y ensembles les da igual; a los lineales con regularización y a los métodos de distancia les resulta imprescindible. Conocer la invariancia del modelo evita trabajo y errores.

4. **"Más features = más información = mejor modelo."** Features irrelevantes añaden varianza y ruido a los métodos de distancia (maldición de la dimensionalidad) y oportunidades de correlación espuria a la selección.
5. **"La correlación baja con el target descarta la feature."** La correlación de Pearson solo ve relaciones lineales marginales: una feature puede ser inútil sola y decisiva en interacción (XOR), o no lineal (información mutua sí la detectaría).

🚀 Del aprendizaje a la operación

En producción las features viven en un *feature store* con definiciones versionadas para que entrenamiento y serving computen exactamente lo mismo (el *training-serving skew* es la avería más común); hay que decidir qué hacer con categorías nunca vistas, monitorear la distribución de cada feature (los nulos que suben del 2 % al 30 % rompen el modelo en silencio), auditar features que son proxies de atributos protegidos (código postal ≈ raza/ingreso en muchos países, ver clase 047), y presupuestar la latencia de features calculadas en tiempo real frente a las precalculadas.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), §7.10.2 "The Wrong and Right Way to Do Cross-validation", PDF oficial — uso: desarrollo extendido del tema
- Guyon & Elisseeff (2003), "An Introduction to Variable and Feature Selection", JMLR 3 (texto completo oficial) — uso: referencia consultada en su fuente original
- Micci-Barreca (2001), "A Preprocessing Scheme for High-Cardinality Categorical Attributes", SIGKDD Explorations. DOI 10.1145/507533.507538 — uso: fuente primaria del mecanismo estudiado
- scikit-learn User Guide — Preprocessing data — uso: referencia consultada en su fuente original
- scikit-learn User Guide — Feature selection — uso: referencia consultada en su fuente original
- scikit-learn User Guide — Pipelines and composite estimators — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P81 · Introducción a la selección de variables y características	2003	Ordena el problema de elegir variables y demuestra por qué el ranking de una en una falla en las dos direcciones.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define ingeniería y selección de características sin usar una marca o framework como definición.
2. Explica la relación entre features, encoding, selección, pipelines.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

043 — Clustering y reducción de dimensionalidad

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **clustering y reducción de dimensionalidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar clustering y reducción de dimensionalidad usando los conceptos `KMeans`, `DBSCAN`, `PCA`, `embeddings`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`KMeans`, `DBSCAN`, `PCA`, `embeddings`

🗺️ Ubicación en el mapa de la IA

Esta clase abandona las etiquetas: el aprendizaje **no supervisado** busca estructura en los datos sin un target que corregir. K-means (Lloyd 1957/1982), el clustering jerárquico y PCA (Pearson 1901, Hotelling 1933) son las herramientas clásicas de exploración, compresión y visualización. Sus ideas — centroides, distancias, proyecciones que preservan varianza — son el vocabulario con el que después se entienden los embeddings (partes 05 y 08): un espacio vectorial donde "cerca" significa "parecido" es exactamente lo que PCA y k-means asumen y lo que las redes aprenden.

📖 Fundamentos

🎯 K-means: minimizar inercia

Dado k (número de clusters), k-means busca centroides μ1..μ_k que minimicen la **inercia** (suma de cuadrados intra-cluster):

```
J = Σᵢ ‖xᵢ − μ_{c(i)}‖²    donde c(i) es el cluster asignado a xᵢ
```

Algoritmo de Lloyd:

```
1. Inicializar k centroides (aleatorio o k-means++)
2. Asignación: cada punto al centroide más cercano
3. Actualización: cada centroide = media de sus puntos
4. Repetir 2-3 hasta que las asignaciones no cambien
```

Cada paso reduce J, así que converge — pero a un **óptimo local** que depende de la inicialización (por eso se corre varias veces; k-means++ elige semillas separadas y mejora la esperanza). Supuestos implícitos: clusters convexos, aproximadamente esféricos y de tamaño similar, en la métrica euclidiana — **escalar features es obligatorio**. Elegir k: método del codo sobre J (heurístico), coeficiente de **silueta** `s = (b−a)/max(a,b)` (a = distancia media intra-cluster, b = distancia media al cluster vecino más próximo; $s \in [-1,1]$, más alto mejor), o conocimiento del dominio.

🌲 Clustering jerárquico

No exige k de antemano: construye un árbol de fusiones (**dendrograma**). El aglomerativo parte de n clusters de un punto y fusiona repetidamente los dos más cercanos según el criterio de enlace (*linkage*):

- **single:** mínima distancia entre puntos — encuentra formas alargadas, sufre encadenamiento;
- **complete:** máxima distancia — clusters compactos;
- **average / Ward:** compromisos; Ward fusiona minimizando el aumento de inercia (el más usado con euclidiana).

Cortar el dendrograma a una altura da una partición; la altura de cada fusión informa cuán "natural" es. Costo $O(n^2)$–$O(n^3)$: inviable para millones de puntos. Alternativa por densidad: **DBSCAN** agrupa puntos con ≥ minPts vecinos en radio ε y marca como ruido lo demás — encuentra formas arbitrarias y outliers, pero sufre con densidades heterogéneas.

📉 PCA: proyectar preservando varianza

PCA busca direcciones ortogonales (componentes principales) que capturan la máxima varianza. Con los datos centrados (media 0), la matriz de covarianza $C = X^\top X/(n-1)$ se descompone en autovectores/autovalores:

$$C \cdot v_j = \lambda_j \cdot v_j \qquad \lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d \geq 0$$

- v_j = dirección del componente j; λ_j = varianza capturada por esa dirección.
- Proyección a m dimensiones: $Z = X \cdot V_m$ (las primeras m columnas de V).
- **Varianza explicada:** $\lambda_j / \Sigma\lambda$; se elige m para retener p. ej. el 90-95 %.
- Equivale a la proyección lineal con mínimo error cuadrático de reconstrucción.
- Requiere centrar; escalar si las unidades difieren (si no, la feature de mayor varianza numérica domina el primer componente).

PCA es lineal y no supervisado: maximiza varianza, que no tiene por qué coincidir con lo discriminativo para una tarea posterior. Para visualización no lineal existen t-SNE y UMAP (preservan vecindades locales, distorsionan distancias globales: sirven para mirar, no para medir).

🧮 Ejemplo trabajado

K-means a mano con 6 puntos en 1D: x = [1, 2, 3, 8, 9, 10], k = 2, centroides iniciales μ1 = 2, μ2 = 3 (mala inicialización a propósito).

```
Iteración 1 — asignación: {1,2 → μ₁}, {3,8,9,10 → μ₂}
             actualización: μ₁ = 1.5, μ₂ = 7.5
Iteración 2 — asignación: |3−1.5|=1.5 < |3−7.5|=4.5 → {1,2,3 → μ₁}, {8,9,10 → μ₂}
             actualización: μ₁ = 2, μ₂ = 9
Iteración 3 — asignaciones no cambian → convergencia
J final = (1+0+1) + (1+0+1) = 4
```

Pese a la mala semilla, aquí converge al óptimo natural. **PCA en 2D:** puntos (2,1), (4,2), (6,3): perfectamente alineados en la dirección (2,1)/√5. λ1 captura el 100 % de la varianza; el segundo autovalor es 0. Proyectar a 1D no pierde nada: los datos eran intrínsecamente unidimensionales.

📊 Propiedades y comparación

Método	k a priori	Forma de clusters	Outliers	Costo	Determinista
k-means (Lloyd)	Sí	Convexos, esféricos	Los absorbe (distorsionan μ)	O(n·k·iter)	No (semilla)
Jerárquico (Ward)	No (dendrograma)	Compactos	Sensible	$O(n^2)$–$O(n^3)$	Sí
DBSCAN	No (ε, minPts)	Arbitraria	Los marca como ruido	O(n log n)	Sí
PCA	m componentes	— (proyección)	Sensible (varianza)	$O(\min(n^2d, nd^2))$	Sí
t-SNE/UMAP	—	— (visualización)	—	Alto	No

⚠️ Errores conceptuales frecuentes

1. **"K-means encontró LOS grupos reales."** K-means siempre devuelve k grupos, haya o no estructura: partirá en k pedazos hasta un gas uniforme. La existencia de clusters se valida (silueta, estabilidad ante re-muestreo), no se asume.
2. **"El codo dice k=4, entonces hay 4 grupos."** La inercia J decrece SIEMPRE con k; el "codo" es una heurística visual frecuentemente ambigua. Contrastar con silueta y con sentido del dominio.

3. **"PCA elige las features importantes."** PCA no elige features: construye combinaciones lineales de todas, ordenadas por varianza — que puede ser ruido de medición. Máxima varianza ≠ máxima relevancia para una tarea supervisada.
4. **"Las distancias del mapa t-SNE se pueden medir."** t-SNE/UMAP preservan vecindades locales y distorsionan lo global: tamaños y separaciones entre islas del gráfico no son evidencia. Para geometría global, PCA.
5. **"No escalé, pero el clustering salió bien."** Sin escalar, la feature de mayor rango domina la distancia euclidiana: el resultado es un clustering de esa feature con ruido del resto — puede "verse bien" y ser un artefacto de unidades.

🚀 Del aprendizaje a la operación

Usar clustering en producción exige: pipeline de escalado idéntico en entrenamiento y scoring, criterio de asignación para puntos nuevos (¿centroide más cercano? ¿re-clustering periódico?), validación de estabilidad (correr con re-muestreos y medir si los grupos persisten — un segmento de clientes que cambia con la semilla no es un segmento), etiquetado humano de los clusters con estadísticas por grupo antes de tomar decisiones, y para PCA, guardar media y componentes exactos del ajuste para transformar el tráfico futuro sin re-ajustar (si no, el espacio cambia bajo el modelo que lo consume).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 14 "Unsupervised Learning", PDF oficial — uso: desarrollo extendido del tema
- James et al. — *An Introduction to Statistical Learning* (2e), cap. 12 "Unsupervised Learning", PDF oficial — uso: desarrollo extendido del tema
- Lloyd (1982), "Least Squares Quantization in PCM", IEEE Trans. Information Theory. DOI 10.1109/TIT.1982.1056489 — uso: fuente primaria del mecanismo estudiado
- Arthur & Vassilvitskii (2007), "k-means++: The Advantages of Careful Seeding", SODA (PDF oficial de Stanford) — uso: referencia consultada en su fuente original
- scikit-learn User Guide — Clustering — uso: referencia consultada en su fuente original
- scikit-learn User Guide — Decomposing signals: PCA — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P53 · Sobre las líneas y planos de ajuste más próximo a sistemas de puntos en el espacio	1901	La primera respuesta al problema de resumir una nube de puntos con menos dimensiones sin privilegiar ninguna variable.	notebook
P73 · Cuantización por mínimos cuadrados en PCM	1982	El algoritmo de agrupamiento más usado del mundo, con la demostración de que converge —y de que converge a un óptimo local, no al global.	notebook
P83 · Visualizar datos con t-SNE	2008	Hace visibles las estructuras locales de datos de alta dimensión, y con ello se convierte en la figura por defecto de media década de artículos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 14 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	citada en las referencias de esta clase · cap. 12 · obra de referencia de la parte 03
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define clustering y reducción de dimensionalidad sin usar una marca o framework como definición.
2. Explica la relación entre KMeans, DBSCAN, PCA, embeddings.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

044 — Detección de anomalías

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **detección de anomalías** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar detección de anomalías usando los conceptos `anomalías`, `aislamiento`, `densidad`, `umbrales`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`anomalías`, `aislamiento`, `densidad`, `umbrales`

🗺️ Ubicación en el mapa de la IA

La detección de anomalías invierte la pregunta habitual del ML: en lugar de modelar lo que las clases tienen en común, modela **lo normal** para señalar lo que se desvía. Es el puente entre el clustering de la clase anterior (densidad, distancia) y aplicaciones donde las etiquetas son escasas por definición — fraude, fallas de máquinas, intrusiones, monitoreo de modelos en producción. Sus tres familias (estadística, densidad, aislamiento) reaparecen después en la observabilidad de sistemas de IA: detectar drift de datos es detectar anomalías sobre las features.

📖 Fundamentos

📏 Enfoque estadístico: z-score y variantes robustas

Si una variable es aproximadamente normal, la desviación estandarizada mide rareza:

```
z = (x − μ) / σ          |z| > 3  →  ~0.3 % de los datos legítimos (regla clásica)
```

Problema: μ y σ se calculan con los mismos datos que contienen las anomalías, y ambas son sensibles a outliers (la anomalía "se esconde" inflando σ: efecto *masking*). Variante robusta con mediana y MAD (desviación absoluta mediana):

```
z_robusto = (x − mediana) / (1.4826 · MAD)      MAD = mediana(|xᵢ − mediana|)
```

El factor 1.4826 hace comparable el MAD con σ bajo normalidad. Límites del enfoque: univariante (o exige modelar la covarianza: distancia de Mahalanobis) y supone una forma de distribución.

🌲 Isolation Forest: aislar en lugar de modelar

Idea (Liu, Ting & Zhou 2008): las anomalías son **pocas y diferentes**, así que son más fáciles de aislar con cortes aleatorios. Se construyen árboles con splits aleatorios (feature aleatoria, corte aleatorio en su rango); la profundidad a la que un punto queda solo (*path length* h(x)) mide su rareza: los outliers se aíslan cerca de la raíz.

```
score(x) = 2^( −E[h(x)] / c(n) )     c(n) ≈ 2·ln(n−1) + 0.5772 − 2(n−1)/n
```

- score → 1: anomalía (camino corto); score ≈ 0.5: punto ordinario.
- c(n) normaliza por la profundidad esperada de un árbol binario de búsqueda con n puntos.
- Complejidad casi lineal, funciona en alta dimensión moderada, no exige distribución.
- Hiperparámetro clave: `contamination` (fracción esperada de anomalías) que fija el umbral sobre el score — es una decisión, no una propiedad de los datos.

🏘️ LOF: densidad local relativa

Local Outlier Factor (Breunig et al. 2000) compara la densidad alrededor de un punto con la de sus k vecinos:

```
LOF(x) ≈ densidad media de los k vecinos / densidad local de x
```

LOF ≈ 1: tan denso como sus vecinos (normal); LOF ≫ 1: mucho menos denso que su entorno (anomalía **local**). Su ventaja distintiva: detecta el punto que es raro *para su región* aunque globalmente parezca ordinario (una compra de 200 USD puede ser normal en una cuenta y anómala en otra). Costo O(n²) ingenuo; sensible a la elección de k.

🎯 Evaluación y decisión

Las anomalías son raras (0.1-5 %): la accuracy es inútil (el baseline "todo normal" acierta 99 %+). Se evalúa con precision-recall sobre las alarmas, y el umbral se fija con la **capacidad de revisión** (si el equipo puede investigar 100 casos/día, el umbral correcto entrega ~100 alarmas/día ordenadas por score) y el costo asimétrico FN/FP (clase 039). Distinguir además: **outlier** (error de dato, se limpia) vs. **anomalía de interés** (fraude, falla: es la señal). El mismo detector encuentra ambos; el triage es humano y de dominio.

🧮 Ejemplo trabajado

Montos de 10 transacciones (USD): [12, 15, 11, 14, 13, 12, 16, 15, 13, 480].

```
Con todo incluido: μ = 60.1, σ ≈ 140  → z(480) = (480−60.1)/140 ≈ 3.0  (apenas dispara)
                                       z(16)  = (16−60.1)/140 ≈ −0.31 (los normales quedan "raros de tan cerca")
Robusto: mediana = 13.5, MAD = mediana(|x−13.5|) = mediana([1.5, 1.5, 2.5, 0.5, 0.5,
         1.5, 2.5, 1.5, 0.5, 466.5]) = 1.5
z_rob(480) = (480 − 13.5) / (1.4826·1.5) ≈ 466.5 / 2.224 ≈ 209.8
```

El z-score clásico casi no detecta el fraude porque el propio fraude infló σ de ~1.5 a 140 (masking); el robusto lo señala con z ≈ 210. Lección: los estimadores del "perfil normal" no deben dejarse contaminar por lo que se busca detectar.

📊 Propiedades y comparación

Método	Supuestos	Tipo de anomalía	Costo	Alta dimensión	Hiperparámetros
z-score / MAD	Distribución unimodal	Global, univariante	O(n)	No (por variable)	Umbral
Mahalanobis	Gaussiana multivariante	Global, correlacionada	O(nd²)	Media	Umbral
Isolation Forest	Pocas y diferentes	Global, multivariante	~O(n log n)	Buena	n árboles, contamination
LOF	Densidad localmente comparable	**Local**	O(n²)	Mala	k, umbral
Basado en clusters	Clusters válidos	Punto lejos de todo centroide	Según método	Media	k del clustering

⚠️ Errores conceptuales frecuentes

1. **"|z|>3 es el umbral universal."** Solo calibra bajo normalidad; con colas pesadas dispara constantemente y con σ contaminada no dispara nunca (masking). El umbral es una decisión operativa, no una constante física.
2. **"El detector encuentra fraudes."** Encuentra *raros*: errores de tipeo, clientes legítimos excéntricos y fraudes por igual. Sin triage humano y feedback etiquetado, la precisión de alarmas se desconoce.
3. **"Accuracy 99.5 %: el detector es excelente."** Con 0.5 % de anomalías, "todo es normal" logra 99.5 %. Las métricas relevantes son precision/recall de la clase rara y alarmas por unidad de revisión.
4. **"Isolation Forest no tiene supuestos."** Supone que las anomalías son pocas y separables por cortes alineados a los ejes; anomalías que solo se ven en combinaciones lineales de features (o

contextuales/temporales) se le escapan.

5. **"Entreno el perfil normal con todos los datos."** Si el histórico ya contiene anomalías, el modelo las aprende como normales. Lo correcto es depurar el train (robustez, revisión) o usar métodos que toleren contaminación declarada.

🚀 Del aprendizaje a la operación

Un detector operativo necesita: bucle de feedback donde cada alarma revisada devuelve una etiqueta (convirtiendo gradualmente el problema en supervisado), recalibración del perfil normal con ventanas móviles (lo normal deriva: estacionalidad, nuevos productos), control de la tasa de alarmas alineado con la capacidad del equipo (alarm fatigue destruye el sistema más preciso), explicación por alarma (qué features dispararon el score, para acelerar el triage) y métricas de negocio — fraude evitado por hora de analista — además de las métricas de la clase rara.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Liu, Ting & Zhou (2008), "Isolation Forest", IEEE ICDM. DOI 10.1109/ICDM.2008.17 — uso: fuente primaria del mecanismo estudiado
- Breunig, Kriegel, Ng & Sander (2000), "LOF: Identifying Density-Based Local Outliers", ACM SIGMOD. DOI 10.1145/342009.335388 — uso: fuente primaria del mecanismo estudiado
- Chandola, Banerjee & Kumar (2009), "Anomaly Detection: A Survey", ACM Computing Surveys. DOI 10.1145/1541880.1541882 — uso: fuente primaria del mecanismo estudiado
- scikit-learn User Guide — Novelty and Outlier Detection — uso: referencia consultada en su fuente original
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), PDF oficial (contexto de densidad y vecinos, cap. 13-14) — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P84 · Bosque de aislamiento	2008	Invierte el planteamiento de la detección de anomalías: en vez de modelar lo normal, mide lo fácil que es aislar cada punto.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 13-14 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define detección de anomalías sin usar una marca o framework como definición.
2. Explica la relación entre anomalías, aislamiento, densidad, umbrales.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;

- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

045 — Series temporales y backtesting

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `ml` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **series temporales y backtesting** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar series temporales y backtesting usando los conceptos `series`, `ventana`, `backtesting`, `forecasting`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`series`, `ventana`, `backtesting`, `forecasting`

🗺️ Ubicación en el mapa de la IA

Las series temporales rompen el supuesto i.i.d. sobre el que descansa todo lo anterior: las observaciones están ordenadas y correlacionadas, y el futuro no está disponible al entrenar. Esta clase adapta el protocolo de la clase 037 al tiempo — el split aleatorio se convierte en **backtesting** con ventanas — e introduce el modelado clásico (tendencia, estacionalidad, ARIMA de Box & Jenkins, 1970). La disciplina de "validar solo hacia adelante" que se aprende aquí es la misma que gobierna la evaluación de agentes y modelos en producción: todo sistema desplegado vive en una serie temporal.

📖 Fundamentos

🧱 Descomposición y estacionariedad

Una serie `y_t` se piensa como composición de **tendencia** (nivel que cambia lento), **estacionalidad** (patrón periódico: semana, año) y **residuo**:

```
aditiva:       y_t = T_t + S_t + e_t      (amplitud estacional constante)
multiplicativa: y_t = T_t · S_t · e_t     (amplitud crece con el nivel → log la vuelve aditiva)
```

Una serie es (débilmente) **estacionaria** si su media, varianza y autocovarianzas no dependen de t. Importa porque los modelos clásicos (AR, MA, ARMA) suponen estacionariedad: sus parámetros son constantes en el tiempo. Herramientas para llegar a ella: **diferenciación** `y'_t = y_t - y_{t-1}` (elimina tendencia; la estacional, `y_t - y_{t-s}`, elimina el patrón de período s) y transformación log (estabiliza varianza). El diagnóstico usa la **ACF** (autocorrelación por rezago): una serie no estacionaria muestra ACF que decae muy lento; los tests formales (ADF, KPSS) contrastan raíz unitaria.

📐 La familia ARIMA (conceptual)

ARIMA(p, d, q) combina tres piezas sobre la serie diferenciada d veces:

```
AR(p):  y_t = c + ϕ₁y_{t-1} + ... + ϕ_p y_{t-p} + e_t     (regresión sobre su pasado)
MA(q):  y_t = c + e_t + θ₁e_{t-1} + ... + θ_q e_{t-q}     (media móvil de errores pasados)
I(d):   aplicar d diferenciaciones antes de modelar
```

Casos límite instructivos: ARIMA(0,1,0) es el **paseo aleatorio** (`y_t = y_{t-1} + e_t`, cuya mejor predicción es el último valor: el baseline *naive*); AR(1) con $|\varphi|<1$ revierte a la media a velocidad φ. La metodología Box-Jenkins: identificar (p,d,q) con ACF/PACF, estimar, validar residuos (deben ser ruido blanco: sin autocorrelación restante) e iterar. SARIMA añade los mismos bloques en el período estacional. En la práctica moderna, ARIMA compite con suavizado exponencial (ETS) y con gradient boosting sobre features de calendario y rezagos (clase 041 + 042).

⏮️ Backtesting: validación que respeta la flecha del tiempo

El split aleatorio mezclaría futuro en el train (fuga temporal). El backtesting simula el uso real: entrenar con el pasado, predecir el futuro inmediato, avanzar y repetir.

```
Origen rodante (rolling origin / walk-forward):
  fold 1: train [1..100]  → test [101..112]
  fold 2: train [1..112]  → test [113..124]   (ventana expansiva)
  fold 3: train [1..124]  → test [125..136]
  ...                                          (ventana deslizante: se descarta lo más viejo)
```

Reglas anti-fuga específicas: los rezagos y estadísticas móviles se calculan solo con datos ≤ t; el escalado se ajusta con cada train del fold; si hay retardo de publicación del dato (la cifra de hoy se conoce mañana), el backtest debe respetarlo; y los **gaps** entre train y test evitan que la autocorrelación de corto plazo filtre información. Métricas: MAE, RMSE, MAPE (cuidado con ceros) y siempre el *skill* relativo al baseline naive o naive estacional: `skill = 1 - MAE_modelo / MAE_naive`.

🧮 Ejemplo trabajado

Serie de ventas semanales: [10, 12, 11, 13, 12, 14, 13, 15]. Comparamos el baseline naive (`ŷ_t = y_{t-1}`) contra una media móvil de 3 (`ŷ_t = media(y_{t-1}, y_{t-2}, y_{t-3})`), prediciendo t = 4..8 (backtest de origen rodante, horizonte 1):

```
t  real  naive  |err|   MM3                    |err|
4  13    11     2       (10+12+11)/3 = 11.00   2.00
5  12    13     1       (12+11+13)/3 = 12.00   0.00
6  14    12     2       (11+13+12)/3 = 12.00   2.00
7  13    14     1       (13+12+14)/3 = 13.00   0.00
8  15    13     2       (12+14+13)/3 = 13.00   2.00
MAE naive = 8/5 = 1.60          MAE MM3 = 6/5 = 1.20
skill = 1 - 1.20/1.60 = 0.25    (la MM3 mejora 25 % al naive)
```

La serie alterna subidas y bajadas (autocorrelación negativa a rezago 1): el naive siempre llega "una semana tarde", la media móvil la suaviza. Primera diferencia: [2, −1, 2, −1, 2, −1, 2] — patrón claro que un AR(1) con φ < 0 modelaría; nada de esto se ve si se evalúa con un split aleatorio.

📊 Propiedades y comparación

Método	Supuestos	Estacionalidad	Exógenas	Interpretación	Cuándo
Naive / naive estacional	Ninguno	Solo el estacional	No	Total	SIEMPRE como baseline
Media móvil / ETS	Nivel/tendencia suaves	ETS sí	No	Alta	Series cortas, univariantes
ARIMA / SARIMA	Estacionariedad tras d	SARIMA	ARIMAX	Media (coeficientes)	Autocorrelación clara
Boosting con rezagos	Features bien hechas	Vía features calendario	Sí, natural	Baja	Muchas series + exógenas

Esquema de validación	¿Respeta el tiempo?	Uso
Split aleatorio / k-fold	❌ (fuga temporal)	Nunca en series
Hold-out temporal único	✅	Chequeo rápido, alta varianza
Rolling origin expansivo	✅	Estándar; usa todo el histórico
Ventana deslizante	✅	Cuando lo viejo ya no representa

⚠️ Errores conceptuales frecuentes

1. **"Hice k-fold sobre la serie y el modelo es buenísimo."** El fold entrenó con el futuro del que predice: fuga temporal pura. En series, la única validación válida avanza en el tiempo.

2. **"R² de 0.98 prediciendo el nivel."** Con series persistentes, predecir `y_{t-1}` ya da R² altísimo. El mérito se mide contra el naive (skill), no contra la media.
3. **"Mi modelo detectó la tendencia" (ajustada sobre toda la serie).** Cualquier suavizado descriptivo (media centrada, filtros bidireccionales) usa datos futuros: sirve para describir, no para predecir ni para generar features.
4. **"Más horizonte, misma confianza."** El error crece con el horizonte (en un paseo aleatorio, la varianza crece linealmente con h); un backtest serio reporta métricas POR horizonte, no promediadas.
5. **"La estacionalidad es obvia, el modelo la aprenderá solo."** Sin el rezago estacional o la feature de calendario, un modelo de rezagos cortos no puede ver el período; el naive estacional lo humilla.

🚀 Del aprendizaje a la operación

Producción añade: re-entrenos programados con ventanas que reflejen la vida útil real de los patrones, monitoreo del error en vivo contra el error prometido por el backtest (si divergen, hubo fuga o el régimen cambió), manejo de datos que llegan tarde o se corrigen (la serie "real" de hace un mes puede cambiar), intervalos de predicción calibrados para decisiones de inventario/capacidad, y detección de cambios de régimen (una pandemia invalida el histórico: saber CUÁNDO tirar datos es tan importante como modelarlos).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("ml")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hyndman & Athanasopoulos — *Forecasting: Principles and Practice* (3e), libro completo gratuito en línea — uso: referencia consultada en su fuente original
- Box, Jenkins, Reinsel & Ljung — *Time Series Analysis: Forecasting and Control* (5e, 2015). DOI 10.1002/9781118619193 (edición 4e) — uso: fuente primaria del mecanismo estudiado
- Bergmeir & Benítez (2012), "On the Use of Cross-validation for Time Series Predictor Evaluation", *Information Sciences*. DOI 10.1016/j.ins.2011.12.028 — uso: fuente primaria del mecanismo estudiado
- scikit-learn — TimeSeriesSplit (validación con orden temporal) — uso: referencia consultada en su fuente original
- statsmodels — Time Series Analysis (ARIMA, descomposición, ACF/PACF) — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P86 · La competición M4: resultados, hallazgos, conclusiones y camino a seguir	2018	Cien mil series y sesenta y un métodos para responder empíricamente qué funciona al predecir series temporales — y la respuesta incomoda a todo el mundo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	obra de referencia de la parte 03 · toda la parte
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define series temporales y backtesting sin usar una marca o framework como definición.
2. Explica la relación entre series, ventana, backtesting, forecasting.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

046 — Sistemas de recomendación

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **sistemas de recomendación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sistemas de recomendación usando los conceptos `filtrado colaborativo`, `contenido`, `ranking`, `cold-start`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`filtrado colaborativo`, `contenido`, `ranking`, `cold-start`

🗺️ Ubicación en el mapa de la IA

Los recomendadores son el sistema de ML clásico con más impacto económico directo: deciden qué ve cada usuario en comercio, streaming y redes. Técnicamente sintetizan casi todo lo anterior — similitud y distancia (clase 043), factorización de matrices (álgebra de la clase 005), ranking y métricas por posición — y son el antecedente conceptual de la recuperación de información que la parte 08 usa en RAG: recomendar es rankear ítems para un usuario, recuperar es rankear documentos para una consulta. El Netflix Prize (2006-2009) hizo de la factorización el estándar del campo.

📖 Fundamentos

🧩 El problema y sus datos

Matriz de interacciones R (usuarios × ítems) casi vacía (típico: > 99 % de celdas desconocidas). Feedback **explícito** (ratings 1-5: escaso y sesgado) o **implícito** (clics, compras, tiempo de reproducción: abundante, pero la ausencia NO es rechazo — solo no exposición). El objetivo real no es rellenar la matriz sino producir un **ranking top-k** útil por usuario.

👥 Filtrado colaborativo por vecindad

"Usuarios que coinciden en el pasado coincidirán en el futuro." Dos variantes:

- **User-based:** buscar usuarios similares al objetivo y promediar sus valoraciones.
- **Item-based** (más estable y usado): similitud entre columnas de R; recomendar ítems similares a los que el usuario ya valoró.

```
similitud coseno:  sim(a, b) = (a·b) / (‖a‖·‖b‖)   sobre las co-valoraciones
predicción:        r̂(u, i) = Σⱼ sim(i, j)·r(u, j) / Σⱼ |sim(i, j)|
```

No necesita features de los ítems; sufre con la dispersión (pocas co-valoraciones → similitudes ruidosas) y no puede recomendar ítems sin historial.

📄 Basado en contenido

Representar cada ítem con sus atributos (género, texto, tags → vectores TF-IDF o embeddings) y recomendar ítems similares al **perfil del usuario** (agregado de lo que consumió). Resuelve el cold-start de ítems nuevos (tienen atributos desde el día cero) y da recomendaciones explicables ("porque viste X"); a cambio, encierra al usuario en más de lo mismo (sobre-especialización) y exige buenos atributos.

🔢 Factorización de matrices

La idea ganadora del Netflix Prize (Koren, Bell & Volinsky 2009): aprender un espacio **latente** de dimensión k donde usuario e ítem son vectores, y la afinidad es su producto interno:

```
r̂(u, i) = μ + b_u + b_i + p_u · q_i
min Σ_{(u,i) observados} ( r(u,i) − r̂(u,i) )² + λ( ‖p_u‖² + ‖q_i‖² + b_u² + b_i² )
```

- μ = media global; b_u, b_i = sesgos de usuario e ítem (un usuario duro, un ítem popular).
- Se optimiza con SGD o ALS **solo sobre las celdas observadas** (no es SVD clásica, que exigiría matriz completa); λ regulariza como en la clase 038.
- Los factores latentes capturan dimensiones no etiquetadas (p. ej. "acción vs. drama") que emergen de los datos: son los embeddings primitivos.
- Para feedback implícito se ponderan las celdas por confianza (Hu, Koren & Volinsky 2008).

📏 Evaluación de rankings y cold-start

El split debe ser **temporal por usuario** (entrenar con su pasado, evaluar su futuro). Métricas top-k: Precision@k y Recall@k (aciertos entre los k recomendados); **NDCG@k** descuenta por posición (`DCG = Σ relᵢ/log₂(i+1)` normalizado por el ranking ideal); cobertura del catálogo y diversidad completan la foto — optimizar solo exactitud produce listas de éxitos idénticas para todos. **Cold-start:** usuario nuevo → contenido, populares, onboarding; ítem nuevo → contenido; sistema nuevo → híbridos. Todo recomendador serio es híbrido: colaborativo cuando hay señal, contenido donde no la hay.

🧮 Ejemplo trabajado

Ratings (1-5) de 4 usuarios sobre 4 películas; ¿qué recomendar a Ana, que no vio D?

```
        A     B     C     D
Ana     5     4     1     ?
Beto    5     5     2     4
Carla   1     2     5     2
Dani    4     4     1     5
```

Similitud coseno de Ana con cada usuario (sobre las columnas A, B, C que comparten):

```
sim(Ana, Beto)  = (25+20+2)/(√42·√54) ≈ 47/47.62 ≈ 0.987
sim(Ana, Carla) = (5+8+5)/(√42·√30)   ≈ 18/35.50 ≈ 0.507
sim(Ana, Dani)  = (20+16+1)/(√42·√33) ≈ 37/37.23 ≈ 0.994
```

Predicción para D ponderando por similitud:

```
r̂(Ana, D) = (0.987·4 + 0.507·2 + 0.994·5) / (0.987 + 0.507 + 0.994)
          = (3.948 + 1.014 + 4.970) / 2.488 ≈ 9.93 / 2.488 ≈ 3.99
```

Se recomienda D (≈ 4/5). Nota el mecanismo: Beto y Dani, con gustos casi idénticos a Ana, dominan la predicción; Carla — de gustos opuestos — apenas pesa. Con similitud de coseno *centrado* (restando la media de cada usuario) Carla incluso restaría, capturando que su 2 en D es, para ella, una valoración relativamente positiva.

📊 Propiedades y comparación

Enfoque	Necesita	Cold-start ítem	Cold-start usuario	Serendipia	Explicable
Colaborativo (vecindad)	Solo interacciones	❌	❌	Alta	Media ("usuarios como tú")
Contenido	Atributos de ítems	✅	Parcial (onboarding)	Baja (burbuja)	Alta ("porque viste X")
Factorización	Muchas interacciones	❌	❌	Alta	Baja (factores latentes)
Popularidad (baseline)	Nada	✅	✅	Nula	Total
Híbrido	Todo lo anterior	✅	✅	Ajustable	Media

⚠️ Errores conceptuales frecuentes

1. **"La celda vacía significa que no le gusta."** En feedback implícito, la ausencia es sobre todo falta de exposición. Tratar los no-vistos como negativos duros sesga el modelo hacia lo ya popular.
2. **"Evalúo con split aleatorio de interacciones."** Mezcla el futuro del usuario en su train (fuga temporal, clase 045) y sobreestima. El split correcto es temporal.
3. **"RMSE bajo = buen recomendador."** El Netflix Prize lo enseñó: optimizar el rating puntual no optimiza el ranking top-k que el usuario ve, ni la diversidad, ni el negocio. Se evalúa con métricas de ranking y A/B.
4. **"El recomendador descubre mis gustos."** También los *crea*: el feedback loop (recomiendo → clic → reentreno con ese clic) amplifica lo popular y estrecha la burbuja; sin corrección de sesgo de exposición, el sistema se retroalimenta.
5. **"Factorización = SVD del álgebra."** La SVD clásica exige matriz completa; aquí se optimiza solo sobre celdas observadas con regularización — un problema distinto, no convexo, resuelto con SGD/ALS.

🚀 Del aprendizaje a la operación

Un recomendador real separa **recall** (candidatos: cientos, con ANN sobre embeddings) de **ranking** (modelo fino sobre los candidatos) por latencia; añade reglas de negocio (stock, edad, ya-comprado), control del feedback loop (exploración: una fracción del tráfico prueba ítems poco expuestos), actualización cercana al tiempo real de los perfiles, y una jerarquía de evaluación en tres niveles — offline (NDCG), online (CTR, conversión) y de largo plazo (retención, diversidad consumida) — porque optimizar el clic de hoy puede degradar el catálogo que el usuario descubre en un año.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.

- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Koren, Bell & Volinsky (2009), "Matrix Factorization Techniques for Recommender Systems", IEEE Computer. DOI 10.1109/MC.2009.263 — uso: fuente primaria del mecanismo estudiado
- Sarwar, Karypis, Konstan & Riedl (2001), "Item-Based Collaborative Filtering Recommendation Algorithms", WWW '01. DOI 10.1145/371920.372071 — uso: fuente primaria del mecanismo estudiado
- Hu, Koren & Volinsky (2008), "Collaborative Filtering for Implicit Feedback Datasets", IEEE ICDM. DOI 10.1109/ICDM.2008.22 — uso: fuente primaria del mecanismo estudiado

- Ricci, Rokach & Shapira (eds.) — *Recommender Systems Handbook* (3e, 2022). DOI 10.1007/978-1-0716-2197-4 — uso: fuente primaria del mecanismo estudiado
- scikit-learn User Guide — Nearest Neighbors (base de la vecindad y similitud) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P85 · Técnicas de factorización matricial para sistemas de recomendación	2009	El método que ganó el Netflix Prize, explicado con lo que de verdad importa: los sesgos antes que los gustos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	obra de referencia de la parte 03 · toda la parte
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define sistemas de recomendación sin usar una marca o framework como definición.

2. Explica la relación entre filtrado colaborativo, contenido, ranking, cold-start.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

047 — Métricas, calibración, sesgo y costo de error

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **métricas, calibración, sesgo y costo de error** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar métricas, calibración, sesgo y costo de error usando los conceptos `métricas`, `calibración`, `fairness`, `costo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`métricas`, `calibración`, `fairness`, `costo`

🗺️ Ubicación en el mapa de la IA

Esta clase cierra el círculo metodológico de la parte 03: después de aprender a entrenar modelos, sistematiza cómo **juzgarlos**. Una métrica mal elegida convierte un buen pipeline en una mala decisión; la calibración conecta scores con probabilidades accionables; el análisis por subgrupos convierte "el modelo funciona" en "¿para quién funciona?". Estas herramientas — matriz de confusión, curvas ROC/PR, Brier, matrices de costo, criterios de equidad — son las mismas con las que después se auditan LLM, agentes y cualquier sistema que decida sobre personas.

📖 Fundamentos

🔢 Matriz de confusión y métricas derivadas

Toda decisión binaria produce cuatro conteos: TP, FP, FN, TN. De ahí:

```
precision = TP/(TP+FP)     "de las alarmas, cuántas eran reales"
recall    = TP/(TP+FN)     "de los casos reales, cuántos atrapé"  (= sensibilidad, TPR)
FPR       = FP/(FP+TN)     "de los negativos, cuántos molesté"
F1        = 2·P·R/(P+R)    media armónica: castiga el desbalance entre P y R
accuracy  = (TP+TN)/n      engañosa con clases desbalanceadas
```

Precision y recall están en tensión: mover el umbral hacia abajo sube recall y baja precision. Cuál priorizar es una decisión de costos, no de estadística.

📈 Curvas: ROC vs. Precision-Recall

Barriendo el umbral se obtienen curvas que evalúan el **score completo**, no una decisión:

- **ROC** (TPR vs. FPR): AUC-ROC = probabilidad de que un positivo al azar reciba score mayor que un negativo al azar (0.5 = azar). Insensible a la prevalencia: útil para comparar modelos, engañosa con clases muy raras (un FPR "bajo" puede ser una avalancha de falsas alarmas en números absolutos).
- **Precision-Recall:** con prevalencia baja es la curva honesta — su baseline es la prevalencia misma, no 0.5. Con 1 % de positivos, un AUC-ROC de 0.95 puede convivir con precision < 0.2 en todo umbral útil.

🌡️ Calibración y Brier

Un score es **calibrado** si $\hat{p}$ = 0.7 implica ~70 % de positivos reales. El **Brier score** mide el error cuadrático de la probabilidad y se descompone (Murphy, 1973):

$$\text{Brier} = \frac{1}{n}\sum(\hat{p}_i - y_i)^2 \;=\; \text{incertidumbre} - \text{resolución} + \text{descalibración}$$

- *Incertidumbre* = $\bar{p}(1-\bar{p})$: piso impuesto por la prevalencia, no reducible.
- *Resolución*: cuánto separan los scores a los grupos con distinta frecuencia real (más es mejor).
- *Descalibración* (reliability): distancia entre score y frecuencia observada (menos es mejor).

Diagnóstico: diagrama de confiabilidad + ECE (error de calibración esperado por bins). Corrección sin reentrenar: Platt o isotónica en validación (clase 039). AUC mide solo *ranking*; Brier/log-loss miden ranking Y calibración: dos modelos con igual AUC pueden diferir radicalmente como estimadores de probabilidad.

💰 Matriz de costos: de métrica a decisión

Asignar costo a cada celda (C_TP, C_FP, C_FN, C_TN) convierte la evaluación en optimización del costo esperado. Con scores calibrados, la decisión óptima por caso es umbral `t* = C_FP/(C_FP + C_FN)` (con C_TP = C_TN = 0); sobre un conjunto:

```
costo total = FP·C_FP + FN·C_FN     → elegir el umbral que lo minimiza en validación
```

Dos modelos deben compararse por costo total con SUS umbrales óptimos respectivos, no por accuracy con t = 0.5.

⚖️ Sesgo y equidad entre subgrupos

Las métricas agregadas ocultan disparidades. Con grupos A y B (género, edad, región…):

- **Paridad demográfica:** misma tasa de predicción positiva por grupo.
- **Igualdad de oportunidades** (Hardt et al. 2016): mismo recall (TPR) por grupo.
- **Igualdad de odds:** mismo TPR y mismo FPR por grupo.
- **Calibración por grupo:** $\hat{p}$ significa lo mismo en A que en B.

Resultado central (Kleinberg et al. 2016; Chouldechova 2017): con prevalencias distintas entre grupos, calibración por grupo e igualdad de tasas de error son **matemáticamente incompatibles** salvo casos triviales (el caso COMPAS: calibrado por raza, pero FPR del doble para acusados negros). Elegir qué criterio priorizar es una decisión normativa que la técnica informa pero no resuelve; el mínimo exigible es **reportar** las métricas por subgrupo.

🧮 Ejemplo trabajado

Screening de una enfermedad con prevalencia 1 %: 10 000 personas, 100 enfermas. Test con recall 90 % y FPR 5 %:

```
TP = 90    FN = 10    FP = 0.05·9900 = 495    TN = 9405
accuracy  = (90+9405)/10000 = 0.9495          ← parece excelente
precision = 90/(90+495) ≈ 0.154               ← 85 % de las alarmas son falsas
```

El baseline "nadie está enfermo" logra accuracy 0.99 — mayor que el test — con recall 0. Con costos C_FN = 1000 (enfermo no detectado) y C_FP = 20 (confirmación innecesaria):

```
costo del test    = 495·20 + 10·1000 = 9900 + 10000 = 19 900
costo del "nadie" = 100·1000         = 100 000
```

El test que la accuracy declaraba "peor que no hacer nada" ahorra el 80 % del costo. Y el Brier de un modelo que asignara $\hat{p} = 0.5$ a todos sería 0.25, mientras el trivial $\hat{p} = 0.01$ constante logra ≈ 0.0099: mejor Brier con cero resolución — por eso la descomposición (y no el número solo) es la lectura correcta.

📊 Propiedades y comparación

Métrica	Evalúa	Sensible a prevalencia	Necesita umbral	Úsala cuando
Accuracy	Decisión	Mucho (engañosa)	Sí	Clases balanceadas, costos simétricos
Precision / Recall / F1	Decisión	Sí (esa es su gracia)	Sí	Clase positiva rara o costosa
AUC-ROC	Ranking	No	No	Comparar scores; cuidado con clase rara
AUC-PR	Ranking	Sí	No	Clase rara: la curva honesta
Log-loss / Brier	Probabilidad	Sí	No	Los scores alimentan decisiones por costo
ECE / confiabilidad	Calibración	Sí	No	Antes de usar t* por costos
Costo esperado	Decisión + negocio	Sí	Sí (t*)	Siempre que haya matriz de costos

⚠️ Errores conceptuales frecuentes

1. **"Accuracy 95 %: excelente modelo."** Con prevalencia 1 %, el baseline trivial da 99 %. Toda métrica se lee contra la prevalencia y el baseline, nunca en absoluto.
2. **"AUC-ROC alto = modelo útil en producción."** El AUC evalúa el ranking global con pares sintéticos; el uso real opera en UN umbral, con prevalencia y costos concretos. Con clase rara, mirar la curva PR y el costo esperado.
3. **"Mis probabilidades salen del modelo, así que son probabilidades."** La salida de `predict_proba` puede estar sistemáticamente desalineada con las frecuencias reales (boosting, redes). Sin verificar calibración, el umbral por costos decide mal.
4. **"El modelo es justo: no usa la variable protegida."** Los proxies (código postal, historial) reconstruyen la variable omitida. La equidad se verifica midiendo por subgrupo, no inspeccionando la lista de features.
5. **"Buscaré un modelo calibrado y con tasas de error iguales entre grupos."** Con prevalencias distintas es imposible (teorema de imposibilidad); hay que elegir qué propiedad priorizar y documentar la decisión.

🚀 Del aprendizaje a la operación

Operar la evaluación implica: intervalos de confianza (bootstrap) sobre cada métrica antes de declarar mejoras, monitoreo de calibración en producción (se degrada con el drift antes que el AUC), revisión periódica de la matriz de costos con el negocio (los costos cambian), paneles por subgrupo con alertas de disparidad, y un registro de decisiones (modelo + umbral + versión de datos) por cada predicción servida — sin esa trazabilidad, ni la auditoría de equidad ni la explicación de un caso individual son posibles.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 7 "Model Assessment and Selection", PDF oficial — uso: desarrollo extendido del tema
- Brier (1950), "Verification of Forecasts Expressed in Terms of Probability", *Monthly Weather Review*. DOI 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2 — uso: fuente primaria del mecanismo estudiado
- Hardt, Price & Srebro (2016), "Equality of Opportunity in Supervised Learning", NeurIPS. arXiv:1610.02413 — uso: fuente primaria del mecanismo estudiado
- Kleinberg, Mullainathan & Raghavan (2016), "Inherent Trade-Offs in the Fair Determination of Risk Scores". arXiv:1609.05807 — uso: fuente primaria del mecanismo estudiado
- Chouldechova (2017), "Fair Prediction with Disparate Impact", *Big Data* 5(2). DOI 10.1089/big.2016.0047 — uso: fuente primaria del mecanismo estudiado
- scikit-learn User Guide — Metrics and scoring — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P80 · Modelización estadística: las dos culturas	2001	Nombra la división que organiza el campo: suponer un mecanismo generador frente a medir la capacidad de predecir.	notebook
P82 · Predecir buenas probabilidades con aprendizaje supervisado	2005	Separa dos cosas que se confundían: ordenar bien los ejemplos y estimar bien la probabilidad de cada uno.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 7 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define métricas, calibración, sesgo y costo de error sin usar una marca o framework como definición.
2. Explica la relación entre métricas, calibración, fairness, costo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

048 — Proyecto: producto ML reproducible

Parte: 03 — Machine learning clásico
Nivel: intermedio · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: producto ml reproducible** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: producto ml reproducible usando los conceptos `pipeline`, `baseline`, `validación`, `serving`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`pipeline`, `baseline`, `validación`, `serving`

🗺️ Ubicación en el mapa de la IA

Este proyecto integra las once clases de la parte 03 en un artefacto único: un pipeline supervisado de punta a punta cuyo valor no es la métrica sino la **reproducibilidad** — que cualquier revisor obtenga el mismo número desde los mismos datos con un comando. Es la transición del ML como experimento al ML como ingeniería (MLOps), y el mismo estándar de evidencia que el programa exigirá después a redes profundas, LLM y agentes: si el resultado no se puede reproducir, no es un resultado.

📖 Fundamentos

📐 Anatomía de un proyecto ML reproducible

```
proyecto/
├── data/           crudos INMUTABLES + procesados regenerables (nunca editar a mano)
├── src/            pipeline: ingesta → features → entrenamiento → evaluación
├── configs/        hiperparámetros y rutas en archivos versionados (no hardcodeados)
├── models/         artefactos entrenados con hash/versión
├── reports/        métricas y figuras GENERADAS por el pipeline
├── tests/          unitarias (features, contratos) + de humo (pipeline end-to-end)
└── README.md       cómo reproducir en UN comando; supuestos y límites
```

Principios: el dato crudo es de solo lectura; todo lo derivado se regenera con código; el pipeline es una función determinista `datos + config + semilla → modelo + métricas`; cada resultado publicado lleva el commit que lo produjo.

🎲 Las cinco fuentes de irreproducibilidad

1. **Datos:** el archivo "final_v2_DEFINITIVO.csv" que nadie sabe cómo se generó. Antídoto: datos crudos inmutables + script de derivación + hash del dataset.
2. **Azar no controlado:** splits, inicializaciones y muestreos sin semilla fija. Antídoto: semillas explícitas en la config; reportar varias semillas (media ± desv.).
3. **Entorno:** versiones distintas de librerías cambian resultados. Antídoto: dependencias con versiones exactas (lockfile) y entorno declarado.
4. **Fuga del protocolo:** decisiones tomadas mirando el test (clases 037 y 042). Antídoto: el pipeline entrena y selecciona sin acceso al test; el test se evalúa en un paso final separado y auditable.
5. **Proceso manual:** "ejecuté las celdas del notebook en cierto orden". Antídoto: pipeline como script/DAG con un solo punto de entrada.

📊 El experimento como contrato

Cada corrida registra: config completa, semilla, hash de datos y código, métricas con baseline, y limitaciones. El resultado se compara SIEMPRE contra el baseline trivial (clase 037) y contra el modelo anterior; una mejora sin intervalo (bootstrap o varias semillas) no es una mejora, es una fluctuación con buena prensa. El informe honesto declara: qué datos, qué protocolo, qué métrica con qué costos, qué subgrupos (clase 047) y qué NO se puede concluir.

🔁 Del notebook al pipeline

El notebook es para explorar; el producto vive en módulos probados:

```
notebook (exploración) → funciones puras en src/ → tests → pipeline CLI → CI
```

Tests mínimos de un proyecto ML: unitarios de features (casos borde, nulos), contrato de esquema de datos (columnas, tipos, rangos), determinismo (misma semilla → mismo resultado), anti-fuga (el preprocesador no ve el test), y humo end-to-end con datos pequeños. La *model card* final documenta uso previsto, datos, métricas por subgrupo y límites — el equivalente ML del `limitations` que este programa exige en cada laboratorio.

🧮 Ejemplo trabajado

Presupuesto de experimento para un clasificador de churn (10 000 clientes, 8 % de bajas):

```
1. Congelar protocolo: split temporal 70/15/15, métrica = costo esperado
   (C_FN = 200 retención perdida, C_FP = 10 llamada), baseline = "nadie se da de baja".
2. Baseline: costo = 800·200 = 160 000 → traducido al split de test (120 bajas): 24 000.
3. Modelo 1 (logística, semillas 1..5): costo test 15 800 ± 900.
4. Modelo 2 (gradient boosting, semillas 1..5): costo test 14 200 ± 1 100.
5. ¿Modelo 2 > Modelo 1? La diferencia (1 600) es mayor que una desviación pero los
   intervalos se solapan → se reporta como "mejora probable, no concluyente"; decisión:
   desplegar logística (más simple, calibrada) y seguir midiendo.
```

La disciplina está en el paso 1 (nada se decide después de ver el test) y en el paso 5 (la conclusión no excede la evidencia — el hábito que este programa entrena desde la clase 008).

📊 Propiedades y comparación

Nivel de madurez	Datos	Código	Experimentos	¿Reproducible?
0: notebook suelto	Archivo local editado	Celdas en orden mental	Ninguno registrado	No
1: scripts + git	Crudos congelados	Versionado	Semilla fija, config en código	A veces
2: pipeline + config	Hash + derivación scriptada	Testeado, CI	Config versionada, métricas emitidas	Sí, en la máquina
3: entorno declarado	Versionados (DVC o similar)	CI + entorno lockeado	Registro por corrida (tracking)	Sí, por terceros

⚠️ Errores conceptuales frecuentes

1. **"El notebook ES el proyecto."** El notebook con estado oculto y orden de ejecución manual es la fuente n.º 1 de resultados no reproducibles; es la herramienta de exploración, no el artefacto final.
2. **"Fijé la semilla, ya es reproducible."** La semilla reproduce UNA realización; si la conclusión cambia con la semilla, lo reproducible es el azar, no el hallazgo. Se reporta sobre varias semillas.
3. **"La métrica subió: despliego."** Sin intervalo, baseline y verificación de que el protocolo no se rompió (¿alguien iteró contra el test?), una subida de métrica es la forma más cara de ruido.
4. **"Reproducible = mismo número exacto siempre."** El estándar práctico es: determinismo dado (datos, config, semilla, entorno), y conclusiones **estables** ante variaciones razonables de semilla y particiones.

5. **"La documentación se escribe al final."** La model card y las limitaciones se llenan durante el desarrollo; al final nadie recuerda qué datos se descartaron ni por qué.

🚀 Del aprendizaje a la operación

Lo que este proyecto educativo aún no cubre y producción exige: registro de experimentos multiusuario (MLflow o equivalente), versionado de datos a escala (DVC, lakehouse), despliegue del artefacto con contrato de entrada/salida y monitoreo de drift + reentrenos (clases 042 y 045), aprobación humana y auditoría para decisiones sensibles (clase 047), y un ciclo de rollback: si el modelo nuevo empeora el costo en producción, volver al anterior debe ser un comando, no una crisis.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Sculley et al. (2015), "Hidden Technical Debt in Machine Learning Systems", NeurIPS 28 (PDF oficial) — uso: referencia consultada en su fuente original
- Mitchell et al. (2019), "Model Cards for Model Reporting", ACM FAT*. DOI 10.1145/3287560.3287596 — uso: fuente primaria del mecanismo estudiado
- Gebru et al. (2021), "Datasheets for Datasets", CACM 64(12). DOI 10.1145/3458723 — uso: fuente primaria del mecanismo estudiado
- Pineau et al. (2021), "Improving Reproducibility in Machine Learning Research", JMLR 22 (texto oficial) — uso: referencia consultada en su fuente original
- scikit-learn — Common pitfalls and recommended practices — uso: referencia consultada en su fuente original
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (2e), cap. 7 (protocolo de evaluación), PDF oficial — uso: desarrollo extendido del tema

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Hastie, Trevor, Tibshirani, Robert y Friedman, Jerome — *The Elements of Statistical Learning*	2.ª · 2009	ISBN 9780387848570 · web de la obra	citada en las referencias de esta clase · cap. 7 · obra de referencia de la parte 03
James, Gareth et al. — *An Introduction to Statistical Learning*	2021	ISBN 9783031387470 · web de la obra	obra de referencia de la parte 03 · toda la parte, nivel introductorio
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 03 · fundamentos probabilísticos del aprendizaje

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: producto ml reproducible sin usar una marca o framework como definición.
2. Explica la relación entre pipeline, baseline, validación, serving.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
